

ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ THÔNG TIN
KHOA KHOA HỌC VÀ KỸ THUẬT THÔNG TIN



**BÁO CÁO CHUYÊN ĐỀ
ĐỒ ÁN TỐT NGHIỆP**

**Đề tài: HỆ THỐNG AGENTIC AI BÁN TỰ ĐỘNG SINH
VÀ KIỂM CHỨNG KIỂM THỬ ĐẦU-CUỐI DỰA TRÊN
PLAYWRIGHT**

*Từ tài liệu yêu cầu đến mã Playwright chạy được,
có truy vết, có neo DOM và có con người phê duyệt*

GVHD: [Họ tên giảng viên hướng dẫn]

Nhóm sinh viên thực hiện:

1. Nguyễn Lê Sang

MSSV: 25210178

2. [Họ tên thành viên 2]

MSSV: [...]

Tp. Hồ Chí Minh, 10/2026

NHẬN XÉT CỦA GIÁO VIÊN HƯỚNG DẪN

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

LỜI MỞ ĐẦU

Kiểm thử đầu-cuối (end-to-end, E2E) kiểm tra một ứng dụng web theo đúng cách người dùng thật sử dụng nó: mở trang, nhập liệu, bấm nút và quan sát kết quả. Viết và duy trì các kịch bản này tốn nhiều công. Kiểm thử viên phải đọc tài liệu yêu cầu, tự suy ra test case, rồi viết mã tự động hoá và sửa lại mỗi khi giao diện thay đổi.

Mô hình ngôn ngữ lớn (LLM) hiện đọc được tài liệu yêu cầu và viết được mã kiểm thử. Khi đưa vào quy trình thật, chúng gặp bốn vấn đề: tài liệu yêu cầu thường mơ hồ nhưng LLM vẫn tự điền chỗ trống; không còn dấu vết nối test với yêu cầu gốc; locator do LLM đoán có thể không tồn tại trên trang; và cơ chế tự sửa lỗi có thể làm yếu assertion để test "xanh" giả. Đồ án này xây dựng một hệ thống Agentic AI bán tự động cho chuỗi từ tài liệu yêu cầu đến mã Playwright Python chạy được. Trong hệ thống, các agent chỉ phân tích, sinh và đề xuất. Con người giữ quyền quyết định ở các công ảnh hưởng tới ý nghĩa kiểm thử.

Nội dung báo cáo gồm sáu chương:

- Chương 1: Tổng quan đề tài
- Chương 2: Cơ sở lý thuyết và nghiên cứu liên quan
- Chương 3: Phân tích, thiết kế và xây dựng hệ thống
- Chương 4: Môi trường thực nghiệm và phương pháp đánh giá
- Chương 5: Kết quả thực nghiệm và đánh giá
- Chương 6: Kết luận và hướng phát triển

Toàn bộ thực nghiệm chỉ chạy trên các website demo công khai được dựng ra để luyện kiểm thử tự động (SauceDemo, The Internet, UI Testing Playground, DemoQA, Practice Test Automation, OrangeHRM Demo). Mọi con số trong báo cáo đều tái lập được bằng lệnh ghi trong Chương 4.

BẢNG PHÂN CÔNG CÔNG VIỆC

Bảng 0.1 Bảng phân công công việc

Thành viên	Vai trò	Công việc chính
Nguyễn Lê Sang	AI / Web / Data	Kiến trúc agent, tích hợp LLM (cloud và local), Requirement Agent, bộ đánh giá RQ1, giao diện Streamlit, thực nghiệm RQ2
[Họ tên thành viên 2]	QA / Playwright	Khảo sát Playwright và locator, tiêu chí đánh giá test case, yêu cầu cho traceability và constrained repair, catalog target RQ2
Cả nhóm		Chốt research gap và MVP, review tài liệu và pull request, ghi biên bản họp

DANH MỤC THUẬT NGỮ

Bảng 0.2 Bảng danh mục thuật ngữ

Thuật ngữ	Giải thích
Kiểm thử đầu-cuối (E2E)	Kiểm thử toàn bộ luồng nghiệp vụ qua giao diện như người dùng thật
Agent (tác tử)	Thành phần dùng LLM để thực hiện một nhiệm vụ có đầu vào, đầu ra theo schema xác định
Locator	Biểu thức xác định phần tử trên trang, ví dụ <code>get_by_role("button", name="Login")</code>
Grounding (neo DOM)	Kiểm chứng locator trên trang thật: phần tử có tồn tại và khớp đúng một hay không
Accessibility tree	Cây trợ năng trình duyệt dựng từ DOM: mỗi nút có role, tên hiển thị và trạng thái
ARIA snapshot	Bản chụp cây trợ năng dạng YAML do Playwright sinh (<code>locator.aria_snapshot()</code>)
Test oracle	Cơ chế quyết định test đạt hay không; ở đây là các <code>assertion expect_*</code>
Self-healing	Tự sửa locator khi phần tử đổi thuộc tính mà nghiệp vụ không đổi
Human-in-the-loop	Con người duyệt tại các điểm quyết định thay vì để hệ thống tự vận hành
Boundary Value Analysis	Kỹ thuật thiết kế test chọn giá trị tại và ngay ngoài biên của ràng buộc
First-run pass rate	Tỉ lệ script do LLM sinh chạy đạt ngay lần đầu, không qua sửa
Over-flag	Tỉ lệ yêu cầu sạch bị gán cờ mơ hồ nhầm
Assertion weakening	Làm yếu lệnh kiểm chứng (ví dụ <code>toBe(5)</code> thành <code>toBeTruthy()</code>) để test đạt giả

DANH MỤC TỪ VIẾT TẮT

Bảng 0.3 Bảng danh mục từ viết tắt

Từ viết tắt	Viết đầy đủ	Ý nghĩa
AG	Approval Gate	Cổng phê duyệt của con người
ARIA	Accessible Rich Internet Applications	Chuẩn W3C mô tả vai trò, tên của phần tử cho công nghệ hỗ trợ
BVA	Boundary Value Analysis	Phân tích giá trị biên
DOM	Document Object Model	Cây đối tượng của trang HTML
DSL	Domain-Specific Language	Tập action Playwright rút gọn mà LLM được phép sinh
E2E	End-to-End	Kiểm thử đầu-cuối
HITL	Human-in-the-loop	Con người trong vòng lặp
LLM	Large Language Model	Mô hình ngôn ngữ lớn
RQ	Research Question	Câu hỏi nghiên cứu
SRS	Software Requirements Specification	Đặc tả yêu cầu phần mềm
TC	Test Case	Cá kiểm thử
UI	User Interface	Giao diện người dùng

DANH MỤC CÁC BẢNG, HÌNH ẢNH

Danh mục các bảng:

Bảng 0.1 Bảng phân công công việc	4
Bảng 0.2 Bảng danh mục thuật ngữ	5
Bảng 0.3 Bảng danh mục từ viết tắt	6
Bảng 1.1 Bốn câu hỏi nghiên cứu và trạng thái trong chuyên đề.....	14
Bảng 1.2 Công nghệ sử dụng	15
Bảng 2.1 Đối chiếu năng lực 12 nguồn với nhu cầu đề tài	20
Bảng 3.1 Các thư mục mã nguồn và vai trò	22
Bảng 3.2 Các trường của StructuredRequirement.....	23
Bảng 3.3 Rubric bảy loại mơ hồ.....	23
Bảng 3.4 DSL action của PlaywrightPlan	24
Bảng 3.5 Phân mức rủi ro của đề xuất sửa	26
Bảng 3.6 Các cổng phê duyệt	27
Bảng 3.7 Các bảng trong cơ sở dữ liệu	28
Bảng 3.8 Ánh xạ đề xuất sửa sang cổng duyệt.....	30
Bảng 4.1 Mười bốn biến thể (mutation) của ShopLab	33
Bảng 4.2 Các website trong catalog RQ2.....	35
Bảng 4.3 Ba nhánh sửa lỗi của RQ3.....	37
Bảng 4.4 Số đo RQ4.....	38
Bảng 5.1 Kết quả phát hiện mơ hồ trên tập 15 yêu cầu.....	39
Bảng 5.2 Ảnh hưởng của việc siết prompt (trước → sau).....	39
Bảng 5.3 F1 theo từng loại mơ hồ	39
Bảng 5.4 Phát hiện mơ hồ theo loại trên 47 yêu cầu (deepseek-chat).....	40
Bảng 5.5 Chất lượng test case theo nhánh và phiên bản quy tắc	41
Bảng 5.6 Test không căn cứ theo loại test (phiên bản 1)	41

Bảng 5.7 First-run pass rate theo website, hai nhánh	42
Bảng 5.8 Bảng chéo kết quả theo cặp target	43
Bảng 5.9 Nguyên nhân thất bại theo nhánh.....	43
Bảng 5.10 Hai lần đo RQ2 trước và sau khi sửa đặc tả goto.....	44
Bảng 5.11 First-run pass rate qua 3 lần lặp (50 target, deepseek-chat).....	45
Bảng 5.12 So sánh theo cặp từng lần lặp.....	45
Bảng 5.13 Kiểm tra assertion rỗng trên các script đã pass	46
Bảng 5.14 Kết quả ba nhánh sửa trên ShopLab (deepseek-chat, ngân sách 2)	47
Bảng 5.15 Kết quả theo mutation (sửa được / escalate / không sửa được)	47
Bảng A.1 Catalog target RQ2 (datasets/reference/rq2_targets.json)	54
Bảng B.1 Bộ test ShopLab (apps/shoplab/suite.py)	58
Bảng C.1 83 điều kiện kiểm thử gold (datasets/reference/ambiguity_eval/test_conditions.json).....	60
Bảng D.1 Lệnh tái lập các số liệu trong báo cáo	63

Danh mục các hình:

Hình 3.1 Kiến trúc tổng quan của hệ thống.....	22
Hình 3.2 Luồng pipeline và các cổng phê duyệt	22
Hình 3.3 Bước ①: yêu cầu trích xuất, cờ mơ hồ và nút duyệt AG-01	29
Hình 3.4 Bước ④: đề xuất sửa từ self-healing, rủi ro Low, chờ người duyệt.....	30
Hình 3.5 Tab Lịch sử: chuỗi truy vết tới script v3 và đề xuất sửa, quyết định append-only	31
Hình 4.1 Quy trình đo RQ2 trên hai nhánh	36
Hình 5.1 Tỷ lệ đạt lần đầu theo website, baseline và DOM-aware.....	43

MỤC LỤC

NHẬN XÉT CỦA GIÁO VIÊN HƯỚNG DẪN	2
LỜI MỞ ĐẦU	3
BẢNG PHÂN CÔNG CÔNG VIỆC	4
DANH MỤC THUẬT NGỮ	5
DANH MỤC TỪ VIẾT TẮT	6
DANH MỤC CÁC BẢNG, HÌNH ẢNH	7
Chương 1: TỔNG QUAN ĐỀ TÀI	13
1.1 Đặt vấn đề và lý do chọn đề tài	13
1.2 Mục tiêu của đề tài	13
1.3 Câu hỏi nghiên cứu	14
1.4 Phạm vi nghiên cứu	14
1.4.1 Trong phạm vi	14
1.4.2 Giới hạn của đề tài	14
1.5 Đóng góp chính	15
1.6 Công nghệ sử dụng	15
Chương 2: CƠ SỞ LÝ THUYẾT VÀ NGHIÊN CỨU LIÊN QUAN	17
2.1 Kiểm thử đầu-cuối và Playwright	17
2.1.1 Kiểm thử đầu-cuối	17
2.1.2 Playwright và chính sách locator	17
2.1.3 Accessibility tree và ARIA snapshot	17
2.2 Mô hình ngôn ngữ lớn và hệ tác tử	18
2.2.1 Structured output	18
2.2.2 Agent và con người trong vòng lặp	18
2.3 Kỹ thuật thiết kế test case	18
2.4 Truy vết yêu cầu	18

2.5 Self-healing và bài toán oracle	19
2.6 Chỉ số đánh giá	19
2.7 Các nghiên cứu liên quan	19
Chương 3: PHÂN TÍCH, THIẾT KẾ VÀ XÂY DỰNG HỆ THỐNG	21
3.1 Yêu cầu hệ thống	21
3.1.1 Yêu cầu chức năng	21
3.1.2 Yêu cầu phi chức năng	21
3.2 Kiến trúc tổng quan	21
3.3 Requirement Analysis Agent.....	23
3.4 Test Design Agent và Boundary Value Analysis	24
3.5 Playwright Generation Agent	24
3.5.1 DSL action rút gọn	24
3.5.2 Chính sách locator và grounding	25
3.5.3 Ngữ cảnh accessibility tree (nhánh DOM-aware)	25
3.5.4 Vòng lặp grounding	26
3.6 Execution Agent	26
3.7 Repair Agent và chính sách sửa có ràng buộc.....	26
3.8 Truy vết, công duyệt và giao diện	27
3.9 Lưu trữ và repository	28
3.10 Giao diện người dùng	28
3.11 Lớp trừu tượng LLM	31
3.12 Kiểm thử phần mềm	32
Chương 4: MÔI TRƯỜNG THỰC NGHIỆM VÀ PHƯƠNG PHÁP ĐÁNH GIÁ	33
4.1 Môi trường thực nghiệm.....	33
4.2 ShopLab: ứng dụng đích có mutation kiểm soát được	33
4.3 Đánh giá RQ1: yêu cầu có cấu trúc	34
4.3.1 Dataset	34

4.3.2 Phát hiện mơ hồ	34
4.3.3 Chất lượng test case: hai nhánh	34
4.4 Đánh giá RQ2: ngữ cảnh DOM và first-run pass rate	35
4.4.1 Catalog target.....	35
4.4.2 Oracle viết tay: kiểm chứng target khả thi	35
4.4.3 Các nhánh so sánh	36
4.4.4 Kiểm soát nhiễu hạ tầng	37
4.5 Đánh giá RQ3: sửa lỗi có ràng buộc	37
4.6 Đánh giá RQ4: thí nghiệm có người tham gia.....	38
Chương 5: KẾT QUẢ THỰC NGHIỆM VÀ ĐÁNH GIÁ	39
5.1 RQ1: yêu cầu có cấu trúc.....	39
5.1.1 Phát hiện mơ hồ trên tập ban đầu	39
5.1.2 Phát hiện mơ hồ trên tập mở rộng	40
5.1.3 Chất lượng test case: pipeline có cấu trúc so với đưa thẳng tài liệu	40
5.2 RQ2: ngữ cảnh DOM và first-run pass rate.....	42
5.2.1 Lần đo thứ nhất: một lỗi đặc tả prompt bị che khuất	43
5.2.2 Thảo luận về lần đo đầu.....	44
5.2.3 Bản 2: grounding vòng lặp, ba lần lặp.....	45
5.3 RQ3: sửa lỗi có ràng buộc	47
5.4 RQ4: công sức thủ công	49
5.5 Môi đe dọa tính hợp lệ.....	49
Chương 6: KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN	51
6.1 Kết quả đạt được.....	51
6.2 Hạn chế	52
6.3 Hướng phát triển.....	52
TÀI LIỆU THAM KHẢO	53
PHỤ LỤC A: DANH SÁCH 50 TARGET RQ2	54

PHỤ LỤC B: BỘ 20 TEST SHOPLAB (RQ3).....	58
PHỤ LỤC C: ĐIỀU KIỆN KIỂM THỬ GOLD (RQ1)	60
PHỤ LỤC D: HƯỚNG DẪN CHẠY LẠI.....	63

Chương 1: TỔNG QUAN ĐỀ TÀI

1.1 Đặt vấn đề và lý do chọn đề tài

Trong quy trình phát triển phần mềm, kiểm thử E2E là chốt chặn cuối trước khi phát hành. Việc chuyển thủ công tài liệu yêu cầu (SRS, use case, user story) thành kịch bản và mã kiểm thử tốn thời gian và khó bảo trì. Một nghiên cứu công nghiệp quy tới 73% thất bại của test suite về lỗi locator chứ không phải lỗi chức năng [6]. Các nghiên cứu 2024–2026 cho thấy LLM sinh được test case từ tài liệu yêu cầu [5][7][11] và thao tác được trên web thật [10]. Khi áp dụng thực tế, nhóm nhận thấy bốn vấn đề mà chưa nguồn nào trong 12 nguồn khảo sát giải quyết trọn vẹn:

- Yêu cầu chưa được quản trị như một artifact có cấu trúc. Các công cụ thường đưa nguyên tài liệu cho LLM, giả định đầu vào đầy đủ và không có bước phát hiện yêu cầu thiếu, mơ hồ hoặc mâu thuẫn [4][7]. LLM vì thế tự điền giả định vào expected result.
- Truy vết bị đứt. Ma trận truy vết thường nằm trong Excel hoặc hệ thống ALM tách khỏi mã, và khoảng 80% người hành nghề coi chi phí cập nhật thủ công là rào cản chính [9]. Khi LLM tự sinh test, không ai biết đoạn mã nào phục vụ yêu cầu nào.
- Locator do LLM đoán có thể không tồn tại. Không neo vào DOM thật, mô hình suy ra selector từ mẫu đã thấy khi huấn luyện; một case study ghi nhận agent bỏ qua công cụ kiểm chứng DOM và gây hàng loạt lỗi timeout [8].
- Tự sửa lỗi không ràng buộc làm hỏng oracle. Trên 300 báo cáo thực thi, một hệ multi-agent chỉ hội tụ thật khoảng 50% thay vì 70% bề mặt, vì agent làm yếu assertion hoặc xóa test đang lỗi để đạt trạng thái Pass [8].

Đề tài chọn hướng bán tự động: LLM làm phần nặng (phân tích, sinh, đề xuất), còn các quyết định làm thay đổi ý nghĩa kiểm thử phải qua con người. Hướng này khớp với khuyến nghị của [8] về giới hạn vòng sửa, phát hiện trôi ngữ nghĩa và bắt buộc phê duyệt thay đổi assertion.

1.2 Mục tiêu của đề tài

- Xây dựng pipeline năm agent: Requirement Analysis, Test Design, Playwright Generation, Execution và Repair, nối từ tài liệu yêu cầu tới script Playwright Python chạy trên website thật.

- Chuẩn hoá yêu cầu thành schema có cấu trúc và gắn cờ mơ hồ theo rubric, chặn ở cổng duyệt trước khi sinh test.
- Duy trì truy vết từ yêu cầu tới test case, script, kết quả chạy và đề xuất sửa.
- Giới hạn tự sửa lỗi bằng chính sách rủi ro tất định (không do LLM tự đánh giá) và ngân sách sửa.
- Đo định lượng: độ chính xác phát hiện mơ hồ (RQ1) và tỉ lệ script chạy đạt ngay lần đầu khi có và không có ngữ cảnh DOM (RQ2).

1.3 Câu hỏi nghiên cứu

Bảng 1.1 Bốn câu hỏi nghiên cứu và trạng thái trong chuyên đề

RQ	Câu hỏi	Trạng thái
RQ1	Chuẩn hoá tài liệu yêu cầu thành biểu diễn có cấu trúc có cải thiện tính hợp lệ và độ bao phủ của test case so với đưa thẳng tài liệu cho LLM không?	Đã đo: 15 tài liệu, 47 yêu cầu, 83 điều kiện gold
RQ2	Kết hợp yêu cầu với DOM hoặc accessibility tree có cải thiện tỉ lệ mã Playwright chạy đạt ở lần đầu không?	Đã đo: 50 target, 6 site, 3 nhánh
RQ3	Sửa lỗi có ràng buộc và phê duyệt của con người có tăng tỉ lệ sửa thành công mà không đổi ý nghĩa test case không?	Đã đo: ShopLab, 14 mutation, 3 nhánh
RQ4	Hệ thống giảm bao nhiêu thời gian và thao tác thủ công so với quy trình truyền thống?	Đã có giao thức và công cụ; chưa có dữ liệu người tham gia

1.4 Phạm vi nghiên cứu

1.4.1 Trong phạm vi

Đề tài nhằm vào kiểm thử chức năng E2E cho ứng dụng web, sinh mã Playwright bằng Python. Đầu vào là tài liệu DOCX, PDF dạng văn bản, Markdown hoặc text, chứa use case, user story kèm acceptance criteria hoặc functional requirement. Hệ thống sinh test case positive, negative, boundary, error guessing và alternative flow; chạy test, thu bằng chứng; đề xuất sửa trong ràng buộc và có con người phê duyệt.

1.4.2 Giới hạn của đề tài

- Không kiểm thử mobile, desktop, hiệu năng, bảo mật hay accessibility toàn diện.
- Không sinh đồng thời Selenium, Cypress và Playwright; chỉ Playwright Python, trình duyệt Chromium, chạy tuần tự.
- Không tự sửa mã nguồn của ứng dụng được kiểm thử và không tự suy ra oracle từ yêu cầu mơ hồ.

- Không huấn luyện hay fine-tune LLM; chỉ dùng mô hình có sẵn qua API tương thích OpenAI.
- DSL chỉ gồm goto, fill, click và ba loại assertion; kịch bản cần chọn dropdown, hover, nhấn phím hay double click nằm ngoài phạm vi.
- Thực nghiệm dùng ứng dụng demo và ứng dụng ShopLab do nhóm tự dựng, không phải ứng dụng doanh nghiệp.

1.5 Đóng góp chính

Đề tài không tuyên bố phát minh từng thành phần riêng lẻ. Phần mới nằm ở cách ghép và kiểm soát chúng trong một pipeline Playwright Python:

- Requirement Analysis Agent với rubric bảy loại mơ hồ, chỉ phát hiện mà không tự giải quyết, chờ duyệt ở AG-01. Cột "ambiguity detection" tổng trên cả 12 nguồn khảo sát.
- Sinh test biên và âm bản tất định bằng Boundary Value Analysis từ ràng buộc số, bổ sung cho kết quả của LLM.
- Grounding đa bước: đi theo luồng và kiểm chứng locator trên DOM tại đúng bước đó, bắt được phần tử chỉ xuất hiện sau điều hướng.
- Repair policy tất định phân mức rủi ro theo nội dung thay đổi (locator, dữ liệu, assertion, bước), ngân sách sửa bằng 2 và self-healing locator từ DOM sống, mọi đề xuất đều cần người duyệt.
- Bộ đánh giá có ground truth: RQ1 trên 47 yêu cầu chèn khuyết tật có chủ đích và 83 điều kiện kiểm thử gold; RQ2 trên catalog 50 target có oracle viết tay chứng minh mỗi target làm được; RQ3 trên ShopLab, một ứng dụng đích có 14 mutation gắn nhãn kỹ thuật/ngữ nghĩa, đo được việc cơ chế sửa có che lỗi thật hay không.
- Hệ thống chạy được trọn luồng trên giao diện, lưu version, quyết định duyệt append-only và chuỗi truy vết vào DB.

1.6 Công nghệ sử dụng

Bảng 1.2 Công nghệ sử dụng

Thành phần	Công nghệ	Vai trò
Ngôn ngữ	Python 3.14	Toàn bộ hệ thống và script kiểm thử sinh ra
Tự động hoá trình duyệt	Playwright 1.62 (sync API), Chromium	Grounding, aria snapshot, chạy script

Thành phần	Công nghệ	Vai trò
Mô hình dữ liệu	Pydantic 2	Schema đầu ra của agent, validate JSON
LLM cloud	DeepSeek (deepseek-chat) qua API tương thích OpenAI	Backend chính cho thực nghiệm
LLM local	LM Studio + google/gemma-4-12b (GPU 8 GB)	Backend miễn phí, chạy offline
Sinh mã	Jinja2	Render PlaywrightPlan thành script Python tự chứa
Đọc tài liệu	python-docx, PyMuPDF	Trích văn bản DOCX/PDF, giữ mốc trang để truy vết
Giao diện	Streamlit	Quy trình duyệt AG-01..05, sinh và chạy script, lịch sử
Lưu trữ	SQLAlchemy 2; SQLite / SQL Server Express	Version, quyết định, truy vết, nhật ký thao tác
Ứng dụng đích	Flask (ShopLab)	App có mutation kiểm soát được cho RQ3, RQ4
Kiểm thử	pytest, pytest-playwright	Unit test offline; chạy test của người tham gia RQ4

Chương 2: CƠ SỞ LÝ THUYẾT VÀ NGHIÊN CỨU LIÊN QUAN

2.1 Kiểm thử đầu-cuối và Playwright

2.1.1 Kiểm thử đầu-cuối

Kiểm thử E2E chạy ứng dụng hoàn chỉnh qua giao diện, mô phỏng hành vi người dùng từ đầu tới cuối một luồng nghiệp vụ. Loại kiểm thử này bắt được lỗi tích hợp mà unit test bỏ sót, đổi lại chậm và dễ gây khi giao diện thay đổi. Mỗi test gồm chuỗi bước (mở trang, nhập, bấm) và một hoặc nhiều assertion đóng vai oracle.

2.1.2 Playwright và chính sách locator

Playwright là thư viện tự động hoá trình duyệt của Microsoft, hỗ trợ Chromium, Firefox và WebKit. Hai cơ chế giúp test ổn định hơn. Thứ nhất là auto-wait: thao tác chờ phần tử sẵn sàng, assertion expect() tự thử lại đến hết timeout. Thứ hai là locator hướng người dùng [2]. Tài liệu chính thức khuyến nghị ưu tiên get_by_role, get_by_label, get_by_placeholder, get_by_text và get_by_test_id, hạn chế CSS/XPath vì chúng bám vào cấu trúc DOM dễ đổi. Locator ở chế độ strict: nếu khớp nhiều hơn một phần tử, thao tác báo lỗi. Hệ thống tận dụng điều này để đo chất lượng locator bằng số phần tử khớp.

Từ 2025, Playwright có bộ Test Agents gồm Planner, Generator và Healer [1]: Planner khám phá ứng dụng và viết kế hoạch Markdown, Generator chuyển thành test, Healer chạy lại và vá test hỏng. Bộ này là baseline sản phẩm gần nhất của đề tài nhưng không có bước phát hiện mơ hồ, không có cổng duyệt bắt buộc và chưa công bố chính sách sửa chi tiết.

2.1.3 Accessibility tree và ARIA snapshot

Trình duyệt dựng accessibility tree từ DOM cho công nghệ hỗ trợ (trình đọc màn hình). Mỗi nút có role (button, textbox, link, heading...), tên hiển thị (accessible name) và trạng thái. Cây này gọn hơn DOM nhiều lần vì bỏ các thẻ trình bày, và nó dùng đúng ngôn ngữ của get_by_role. Playwright từ bản 1.49 có locator.aria_snapshot() trả cây này dạng YAML, ví dụ:

```
- heading "Dynamic Controls" [level=4]
- checkbox
- text: A checkbox
- button "Remove"
```

```
- textbox [disabled]
- button "Enable"
```

Đây là ngữ cảnh DOM mà nhánh DOM-aware của RQ2 đưa cho LLM. Hướng ally-first cũng là lỗi của [6], còn [10] lọc DOM rồi đánh chỉ số phần tử cho LLM chọn.

2.2 Mô hình ngôn ngữ lớn và hệ tác tử

2.2.1 Structured output

LLM sinh văn bản tự do, trong khi pipeline cần dữ liệu có cấu trúc để validate và lưu. Các API tương thích OpenAI hỗ trợ response_format kiểu json_schema: máy chủ ép đầu ra theo ngữ pháp sinh từ schema, nên mô hình không thể trả sai kiểu hay thiếu trường. Hệ thống khai báo schema bằng Pydantic, gửi kèm response_format và validate lại phía client.

2.2.2 Agent và con người trong vòng lặp

Trong đồ án, agent là một bước có đầu vào và đầu ra theo schema, dùng LLM ở phần cần suy luận và dùng code tắt định ở phần còn lại. Mô hình human-in-the-loop đặt con người tại các điểm có rủi ro ngữ nghĩa: duyệt yêu cầu đã chuẩn hoá, duyệt test case, duyệt mọi đề xuất sửa đổi assertion, expected result hay bước kiểm thử. Bằng chứng ở [8] cho thấy thiếu các công này thì agent tối ưu cho trạng thái Pass thay vì cho tính đúng.

2.3 Kỹ thuật thiết kế test case

Kiểm thử hộp đen dựa trên đặc tả có các kỹ thuật kinh điển trong chuẩn ISO/IEC/IEEE 29119-4: phân hoạch tương đương, phân tích giá trị biên, đoán lỗi và kiểm thử luồng thay thế. Với ràng buộc "mật khẩu 8–20 ký tự", BVA chọn các giá trị 8 và 20 (biên hợp lệ) cùng 7 và 21 (ngay ngoài biên, ca âm bản). [7] dùng các kỹ thuật này trong prompt; đồ án tách BVA ra thành code tắt định để bảo đảm luôn có ca biên, không phụ thuộc LLM nhớ ra.

2.4 Truy vết yêu cầu

Requirements traceability là khả năng đi xuôi từ một yêu cầu tới các artifact hiện thực nó và đi ngược lại. ReqToCode [9] nhúng phần tử truy vết vào mã để lỗi truy vết thành lỗi biên dịch, nhưng chỉ bảo đảm sự hiện diện cấu trúc chứ không bảo đảm đúng ngữ nghĩa, và không bao phủ kết quả chạy hay đề xuất sửa. Đồ án dùng chuỗi ID tắt định

REQ-001 → REQ-001-TC-01 → script → kết quả → đề xuất sửa để tính độ phủ và tìm artifact mô côi.

2.5 Self-healing và bài toán oracle

Self-healing locator tìm phần tử thay thế khi locator cũ gãy. Similo và HybridSimilo [12] chấm điểm ứng viên trên nhiều thuộc tính (tag, id, class, text, aria-label, vị trí) và đặt tỉ lệ định vị lại cao trên benchmark của họ. [6] chỉ trích lại đúng selector hỏng từ accessibility tree, không tốn lời gọi API. Các kỹ thuật này sửa được phần kỹ thuật nhưng không trả lời được câu hỏi khi nào thay đổi là hợp lệ về nghiệp vụ. Khi agent được phép sửa cả assertion, nó có thể sửa oracle cho khớp với lỗi [8]. Ranh giới mà đồ án chọn: locator tương đương là thay đổi kỹ thuật; assertion, expected result và bước kiểm thử là thay đổi ngữ nghĩa và phải qua người duyệt.

2.6 Chỉ số đánh giá

- Precision = $TP / (TP + FP)$: trong các cờ mờ hồ hệ thống gắn, bao nhiêu phần đúng. Recall = $TP / (TP + FN)$: trong các khuyết tật thật, hệ thống bắt được bao nhiêu. F1 là trung bình điều hoà của hai chỉ số.
- Micro-F1 gộp TP/FP/FN của mọi loại rồi tính; macro-F1 tính F1 từng loại rồi lấy trung bình, nên loại hiếm có trọng số ngang loại phổ biến.
- Over-flag: tỉ lệ yêu cầu sạch (không có khuyết tật chèn) bị gắn cờ. Chỉ số này đo gánh nặng duyệt nhằm cho con người.
- First-run pass rate: số script đạt ngay lần chạy đầu chia cho số target. Script không được sửa tay hay qua Repair Agent.
- Kiểm định McNemar: hai nhánh chạy trên cùng target nên kết quả là dữ liệu cặp. McNemar chỉ dùng các cặp bất đồng (nhánh này đạt, nhánh kia trượt); với cỡ mẫu nhỏ dùng dạng chính xác theo phân phối nhị thức.

2.7 Các nghiên cứu liên quan

Nhóm khảo sát 12 nguồn chia năm mảng: baseline sản phẩm, requirement → test bằng LLM, traceability, DOM grounding và self-healing có ràng buộc. Bảng 2.1 tóm tắt năng lực của các nguồn theo tám tiêu chí mà đề tài cần.

Bảng 2.1 Đối chiếu năng lực 12 nguồn với nhu cầu đề tài

Nguồn	Yêu cầu cấu trúc	Phát hiện mơ hồ	Truy vết	Neo DOM	Oracle	Duyệt người	Sửa ràng buộc
[1] Playwright Test Agents	Một phần	Không	Một phần	Có	Một phần	Không	Một phần
[2] Codegen/locator	Không	Không	Không	Có	Một phần	Một phần	Không
[3] RBTG Survey	Một phần	Một phần	Một phần	Không	Một phần	Một phần	Không
[4] WebTestPilot	Có	Không	Có	Có	Có	Không	Một phần
[5] LLM Scenario Tool	Một phần	Không	Không	Không	Một phần	Một phần	Không
[6] Zero-Cost Self-Healing	Không	Không	Một phần	Có	Một phần	Không	Có
[7] GHF (ISO 29119-4)	Một phần	Không	Một phần	Không	Một phần	Một phần	Không
[8] Autonomous Repair Limits	Một phần	Không	Một phần	Một phần	Không	Một phần	Có
[9] ReqToCode	Có	Không	Có	Không	Không	Một phần	Không
[10] Steward	Không	Không	Không	Có	Một phần	Không	Một phần
[11] System TC (ChatGPT)	Có	Không	Một phần	Không	Một phần	Có	Không
[12] Web Element Relocalization	Không	Không	Không	Có	Không	Không	Có
Nhu cầu đề tài	Có	Có	Có	Có	Có	Có	Có

Cột phát hiện mơ hồ không có nguồn nào đạt "Có". [3] chỉ nêu đây là thách thức mở; [4] giả định đặc tả tự chứa và đầy đủ; [11] ghi nhận LLM hiểu hành vi hệ thống hạn chế khi chỉ đọc SRS nhưng không có bộ phát hiện. Không nguồn nào phủ đủ tám tiêu chí. Mỗi nguồn mạnh ở vài mảnh, nên khoảng trống mà đề tài nhắm tới là sự tích hợp có kiểm soát: một pipeline bán tự động có truy vết, có neo DOM, và ngăn self-healing làm đổi oracle khi chưa được duyệt.

Về RQ2, [10] báo cáo tỉ lệ hoàn thành tác vụ end-to-end 40% với GPT-4 khi có lọc DOM và screenshot, còn [8] cho thấy chiều ngược lại: khi agent bỏ qua kiểm chứng DOM, selector ảo giác gây lỗi dây chuyền. Chưa nguồn nào trong bộ khảo sát so sánh có kiểm soát cùng một tập test case khi có và không có ngữ cảnh accessibility tree. Thực nghiệm RQ2 của đồ án làm đúng phép so sánh này.

Chương 3: PHÂN TÍCH, THIẾT KẾ VÀ XÂY DỰNG HỆ THỐNG

3.1 Yêu cầu hệ thống

3.1.1 Yêu cầu chức năng

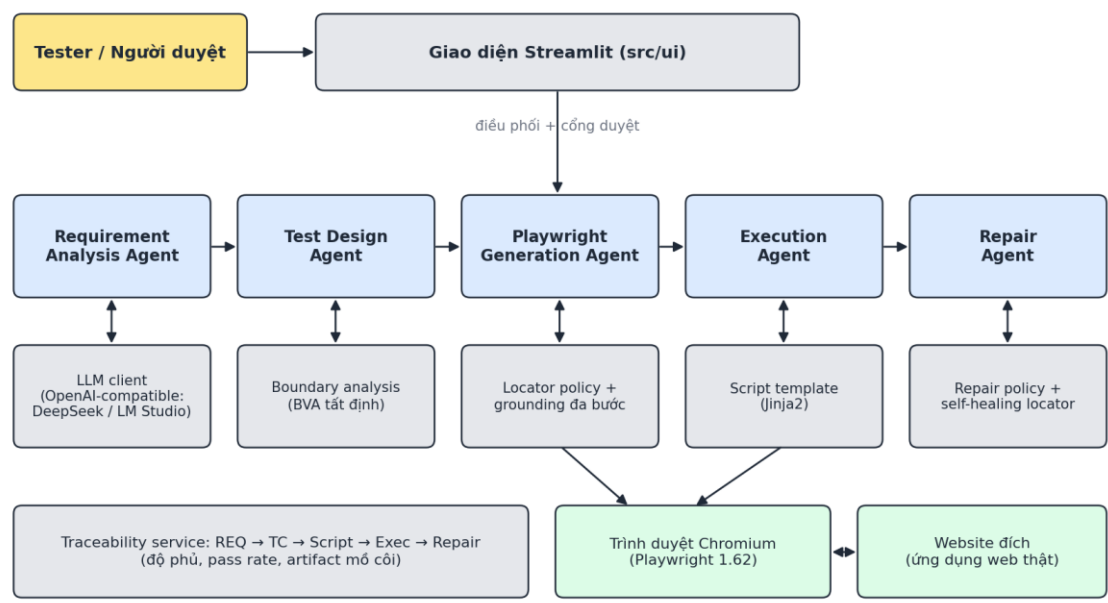
- Nạp tài liệu DOCX, PDF, Markdown, text; trích yêu cầu có cấu trúc kèm trích đoạn nguồn.
- Gắn cờ yêu cầu thiếu, mơ hồ, mâu thuẫn và chờ người duyệt trước khi sinh test (AG-01).
- Sinh test case nhiều loại từ yêu cầu đã duyệt, cho người duyệt hoặc loại bỏ (AG-02).
- Sinh script Playwright Python từ test case, kiểm chứng locator trên trang thật.
- Chạy script, thu ảnh chụp, phân loại passed / failed / error / blocked.
- Khi test lỗi: phân mức rủi ro đề xuất sửa, tự chữa locator từ DOM sống, chờ người duyệt (AG-03..05).
- Dựng chuỗi truy vết, tính độ phủ yêu cầu, độ phủ thực thi, pass rate và artifact mô côi.

3.1.2 Yêu cầu phi chức năng

- Fail-closed: đầu ra sai schema, thiếu bằng chứng hay còn mơ hồ chưa xử lý thì dừng ở trạng thái chờ duyệt.
- Đổi backend LLM (cloud hoặc local) chỉ bằng cách đổi file cấu hình, không sửa agent.
- Kiểm thử offline: unit test không cần mạng, trình duyệt hay API key.
- Không lưu khoá API trong mã nguồn; .env và .env.cloud bị loại khỏi git.
- Tái lập được: mọi số liệu báo cáo chạy lại được bằng lệnh có ghi trong tài liệu.

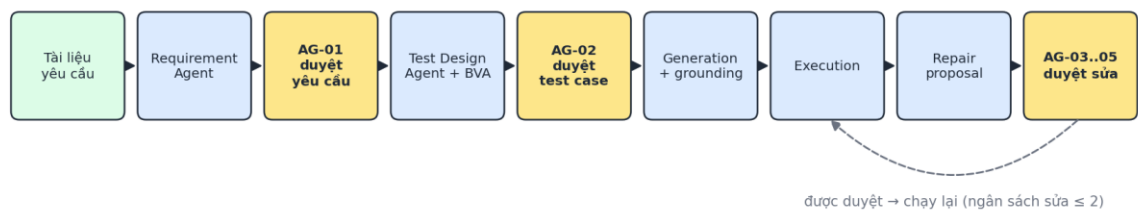
3.2 Kiến trúc tổng quan

Hệ thống gồm năm agent nối tiếp, mỗi agent dựa trên một dịch vụ tắt định (Hình 3.1). Các agent không gọi nhau trực tiếp mà trao đổi qua các model Pydantic. Approval không phải công cụ mà agent tự gọi được: quyết định duyệt chỉ đến từ người dùng qua giao diện. Thiết kế này tránh việc agent tự hợp thức hoá đề xuất của chính nó.



Hình 3.1 Kiến trúc tổng quan của hệ thống

Luồng xử lý đầy đủ đi qua các cổng duyệt như Hình 3.2. Khi test lỗi, Repair Agent tạo đề xuất; đề xuất được duyệt thì script mới được chạy lại; vượt ngân sách sửa thì chuyển sang trạng thái chờ người xem xét.



Ô vàng = cổng con người quyết định; agent chỉ để xuất, không tự áp dụng thay đổi.
Mỗi artifact mang ID truy vết REQ-xxx → REQ-xxx-TC-yy → script → kết quả → để xuất sửa.

Hình 3.2 Luồng pipeline và các cổng phê duyệt

Bảng 3.1 Các thư mục mã nguồn và vai trò

Thư mục / tệp	Vai trò
src/models/	Model Pydantic: Requirement, TestCase, PlaywrightPlan, ExecutionResult, RepairProposal, Approval
src/agents/	Năm agent: requirement, test_design, playwright_generation, execution, repair
src/services/	Dịch vụ tất định: llm_client, boundary_analysis, locator_policy, locator_healing, repair_policy, script_template, traceability, document_loader
src/services/browser_tasks.py	Tác vụ trình duyệt dùng chung: snapshot, grounding, dò bước hỏng, chữa theo lô
src/database/	Schema SQLAlchemy và repository (version, duyệt, truy vết, nhật ký)

Thư mục / tệp	Vai trò
src/repositories/	
src/evaluation/	Bộ đánh giá phát hiện mơ hồ: dataset, metrics, evaluator, design_metrics
src/ui/	Giao diện Streamlit (app.py) và logic duyệt thuần (review.py)
apps/shoplab/	Ứng dụng đích ShopLab, 14 mutation có nhãn, bộ 20 test
run_*.py, evaluate_*.py	CLI cho từng agent, pipeline đầy đủ và các thực nghiệm RQ1–RQ3
tools/	Công cụ thí nghiệm RQ4 và bộ dựng báo cáo
datasets/reference/	Ground truth RQ1 (15 tài liệu), catalog 50 target RQ2, nhiệm vụ RQ4
tests/unit/	Unit test offline

3.3 Requirement Analysis Agent

Agent nhận văn bản do bộ đọc tài liệu trích ra. Với PDF, bộ đọc chèn mốc [trang N] để mỗi yêu cầu truy được về trang gốc. LLM trả về danh sách StructuredRequirement theo schema ở Bảng 3.2. Hệ thống tự đánh ID REQ-001, REQ-002... theo thứ tự và bỏ qua ID do LLM đặt, vì ID của LLM không ổn định giữa các lần chạy. Tài liệu rỗng trả kết quả rỗng mà không gọi LLM.

Bảng 3.2 Các trường của StructuredRequirement

Trường	Ý nghĩa
id	REQ-xxx do hệ thống gán, dùng làm gốc truy vết
title, actor, action	Tên yêu cầu, tác nhân thực hiện, hành động
precondition, expected_outcome	Điều kiện trước và kết quả mong đợi; để trống nếu tài liệu không nói
constraints	Ràng buộc (độ dài, khoảng giá trị...), đầu vào cho BVA
source_excerpt	Trích đoạn nguồn nguyên văn
ambiguities	Danh sách AmbiguityFinding: loại, mô tả, trích đoạn, gợi ý, mức độ

Rubric mơ hồ có bảy loại (Bảng 3.3). Mâu thuẫn giữa nhiều yêu cầu được đặt ở global_ambiguities vì nó không thuộc riêng yêu cầu nào. Agent chỉ phát hiện, không tự làm đầy chỗ thiếu: yêu cầu bị gán cờ dừng ở AG-01 cho tới khi người dùng làm rõ hoặc chấp nhận.

Bảng 3.3 Rubric bảy loại mơ hồ

Loại	Dấu hiệu	Ví dụ
missing_actor	Không rõ ai thực hiện	"Đơn hàng được hủy trong 24 giờ"
missing_precondition	Thiếu điều kiện trước cần thiết	Đổi mật khẩu mà không nói phải đăng nhập
vague_quantifier	Lượng từ không đo được	"phản hồi nhanh", "một vài sản"

Loại	Dấu hiệu	Ví dụ
		phẩm"
missing_expected_outcome	Không nói kết quả mong đợi	"Người dùng bấm Thanh toán" rồi dừng
underspecified_action	Hành động thiếu tham số	"Nhập thông tin hợp lệ"
conflict	Hai yêu cầu mâu thuẫn	"mật khẩu 8–20 ký tự" và "tối thiểu 18 ký tự"
other	Mơ hồ khác	

Một lỗi đáng ghi lại: khi bỏ phần mô tả schema khỏi prompt để tiết kiệm khoảng 35% token đầu vào, Gemma dồn mọi cờ mơ hồ vào `global_ambiguities` thay vì gắn theo từng yêu cầu. Micro-F1 rơi xuống 0,07. Thêm một dòng chỉ dẫn gắn cờ theo từng yêu cầu đưa micro-F1 lên 0,73. Bộ đánh giá định lượng là thứ làm lộ ra lỗi này; đọc mắt kết quả JSON thì khó thấy.

3.4 Test Design Agent và Boundary Value Analysis

Agent chỉ nhận yêu cầu đã duyệt. LLM sinh test case các loại positive, negative, boundary, `error_guessing` và `alternative_flow`, mỗi test case mang ID REQ-xxx-TC-yy. Prompt cấm tạo expected result vượt quá yêu cầu đã duyệt.

LLM hay quên ca biên, nên hệ thống bổ sung module `boundary_analysis` tắt định. Module nhận ràng buộc số tường minh (khoảng, tối thiểu, tối đa) viết bằng tiếng Việt, tiếng Anh hoặc ký hiệu. Từ mỗi ràng buộc, nó sinh ca biên hợp lệ và ca ngay ngoài biên, kèm trích dẫn ràng buộc nguồn. Ràng buộc không rõ thì không sinh (fail-closed). Với "Mật khẩu 8-20 ký tự" và "tối thiểu 18", module sinh 6 ca biên và âm bản đúng nhãn.

3.5 Playwright Generation Agent

3.5.1 DSL action rút gọn

LLM không viết mã Python trực tiếp. Nó trả về một `PlaywrightPlan` gồm danh sách action trong một DSL nhỏ (Bảng 3.4); mã được render từ plan qua template Jinja2. Cách này giới hạn những gì LLM có thể làm, cho phép chính sách repair so sánh hai plan theo từng action, và loại bỏ lỗi cú pháp.

Bảng 3.4 DSL action của `PlaywrightPlan`

Action	Trường dùng	Mã Playwright sinh ra
goto	arg = URL	page.goto(url)

Action	Trường dùng	Mã Playwright sinh ra
fill	strategy, value, arg	<locator>.fill(arg)
click	strategy, value, role_name	<locator>.click()
expect_url	arg	expect(page).to_have_url(arg)
expect_visible	strategy, value, role_name	expect(<locator>).to_be_visible()
expect_text	strategy, value, arg	expect(<locator>).to_contain_text(arg)
(mọi locator)	$nth \geq 0$, tùy chọn	<locator>.nth(nth) khi nhiều phần tử giống hệt nhau

Strategy là một trong role, label, placeholder, text, test_id, css. Với role, value là vai trò ARIA còn role_name là tên hiển thị. Prompt nhấn mạnh điểm này vì lỗi phổ biến nhất của LLM là đặt tên nút vào value. Trường nth được thêm sau khi RQ2 cho thấy một nhóm lỗi không giải được trong DSL cũ: trang có nhiều phần tử cùng vai trò và tên (sáu nút "Add to cart" của SauceDemo).

3.5.2 Chính sách locator và grounding

Khi có nhiều locator ứng viên cho cùng phần tử, locator_policy chọn locator khớp đúng một phần tử, ưu tiên theo thứ tự role, label, placeholder, text, test_id, css như khuyến nghị của Playwright. Grounding đếm số phần tử khớp với mỗi locator trên trang thật và lưu thành GroundingRecord (ok khi khớp đúng 1).

Grounding tĩnh chụp DOM một lần nên bỏ sót phần tử chỉ có sau điều hướng, ví dụ danh sách sản phẩm chỉ hiện sau khi đăng nhập. Hàm ground_flow đi theo luồng: ở mỗi bước, nó chờ locator gắn vào DOM, đếm số phần tử khớp, rồi mới thực hiện goto, fill hoặc click để đẩy trang sang trạng thái kế. Assertion không làm đổi trạng thái. Test case đăng nhập SauceDemo đạt grounding 4/4, kể cả .inventory_list chỉ xuất hiện sau đăng nhập.

3.5.3 Ngữ cảnh accessibility tree (nhánh DOM-aware)

Để trả lời RQ2, hàm plan nhận thêm tham số tùy chọn page_context. Khi có, prompt được nối thêm aria snapshot của trang đích lúc vừa mở, cắt tối đa 6.000 ký tự, kèm chỉ dẫn: với phần tử có trong snapshot thì dùng đúng role và tên như snapshot; phần tử xuất hiện sau điều hướng thì suy luận như thường. Khi không truyền page_context, prompt giữ nguyên như baseline. Unit test kiểm tra cả hai trường hợp.

3.5.4 Vòng lặp grounding

Snapshot cho LLM biết phần tử nào tồn tại nhưng không cho biết locator nó viết ra có khớp đúng một phần tử hay không. Vòng lặp grounding ghép hai cơ chế: sau khi LLM sinh plan, ground_flow chạy plan trên trang thật và đếm số phần tử khớp ở từng bước. Nếu có locator khớp 0 hoặc nhiều hơn 1, hàm grounding_feedback dừng phản hồi gồm action lỗi, số phần tử khớp và aria snapshot của trang tại đúng bước đó (kể cả trang sau điều hướng), rồi gửi lại cho LLM để sinh lại đúng một lần. Script cuối cùng vẫn chỉ được chạy một lần.

3.6 Execution Agent

Execution Agent ghi script ra tệp, chạy trong tiến trình con với timeout 120 giây và thu stdout, stderr, ảnh chụp. Trạng thái được phân loại: passed khi tiến trình thoát mã 0 và in PASSED; failed khi stderr chứa AssertionError, TimeoutError hoặc expect(; blocked khi quá timeout; các trường hợp còn lại là error. Agent không sửa test.

Template script cấu hình các tham số chạy, áp dụng như nhau cho mọi script. Timeout của expect là 10 giây vì một số trang demo cần khoảng 5 giây mới hiện kết quả, đúng bằng mặc định 5 giây của Playwright. Timeout thao tác (click, fill) là 10 giây thay cho mặc định 30 giây, để script hỏng theo chủ đích trong RQ3 không chiếm quá nhiều thời gian; timeout điều hướng giữ 30 giây. Chromium chạy với cờ --disable-http2, lý do trình bày ở mục 4.4.4.

3.7 Repair Agent và chính sách sửa có ràng buộc

Repair Agent nhận test lỗi kèm bằng chứng và tạo đề xuất sửa dạng plan mới. Mức rủi ro do repair_policy quyết định bằng cách so sánh plan cũ và plan mới theo từng action, không để LLM tự khai (Bảng 3.5). Mọi đề xuất đều có requires_approval = true; agent không tự áp dụng. Ngân sách sửa mặc định là 2 lần cho một chuỗi thực thi; vượt ngân sách thì chuyển BLOCKED_FOR_REVIEW.

Bảng 3.5 Phân mức rủi ro của đề xuất sửa

Mức	Thay đổi phát hiện được	Xử lý
Low	Chỉ đổi locator	Đề xuất, người duyệt đơn giản
Medium	Đổi dữ liệu nhập, URL điều hướng, thêm bước	Bắt buộc xem bằng chứng và duyệt
High	Đổi assertion, đổi loại action, bỏ bước	Bắt buộc duyệt, gắn cờ thay đổi ngữ nghĩa

Mức	Thay đổi phát hiện được	Xử lý
Prohibited	Sửa mã nguồn website, xoá bằng chứng, lập vô hạn	Từ chối

Self-healing locator là đường sửa không cần LLM. Khi locator gãy, hệ thống quét DOM sống lấy các ứng viên, so khớp mờ với ý định của locator cũ, đếm số phần tử khớp, rồi chọn ứng viên duy nhất theo locator_policy. Không có ứng viên duy nhất thì trả về rỗng. Trong demo, locator cố ý sai placeholder:Username được chữa thành role:textbox[Username] với rủi ro Low, và vẫn chờ người duyệt.

Lần chạy RQ3 đầu cho thấy một giới hạn: self-healing chỉ chữa locator hỏng đầu tiên mỗi lượt, nên mutation đổi id của cả ba ô đăng nhập cần ba lượt và vượt ngân sách 2. Hàm heal_all chữa theo lô: dò bước hỏng, chữa, chạy lại plan đã chữa tới bước hỏng kế tiếp, lặp tới khi không còn locator chữa được; mọi chỗ chữa gộp thành một đề xuất. Mức rủi ro vẫn do policy quyết định trên toàn bộ thay đổi. Tập ứng viên của thao tác click được mở rộng thêm input[type=submit] và input[type=button], vì nút Login của SauceDemo là một thẻ input.

RepairProposal mang theo plan mới (new_plan) bên cạnh diff. Đề xuất chỉ được áp dụng sau khi người duyệt đồng ý; khi đó plan mới được lưu thành version script mới và chạy lại.

3.8 Truy vết, công duyệt và giao diện

Traceability service dựng chuỗi Requirement → TestCase → Script → Execution → Repair từ các ID, tính độ phủ yêu cầu (tỉ lệ yêu cầu có ít nhất một test case), độ phủ thực thi, pass rate và liệt kê artifact mờ côi. Bảng 3.6 liệt kê các công duyệt.

Bảng 3.6 Các công phê duyệt

Công	Khi nào	Trạng thái hiện thực
AG-01	Yêu cầu bị gắn cờ mơ hồ	Có trên giao diện Streamlit
AG-02	Trước khi sinh script từ test case	Có trên giao diện Streamlit
AG-03	Đề xuất sửa đổi phần tử mà assertion kiểm	Có trên giao diện Streamlit
AG-04	Đề xuất đổi giá trị kỳ vọng (oracle)	Có trên giao diện Streamlit
AG-05	Đề xuất bỏ, thêm hoặc đổi bước kiểm thử	Có trên giao diện Streamlit

3.9 Lưu trữ và repository

Lớp lưu trữ dùng SQLAlchemy 2, chạy trên SQLite khi phát triển và kiểm thử, và trên SQL Server Express khi demo; đổi giữa hai DB chỉ cần biến môi trường DATABASE_URL. Schema bám các entity trong kiến trúc (Bảng 3.7) theo ba nguyên tắc. Một là artifact có thể sửa sau review (yêu cầu, test case, script) có cột version: lưu lại cùng nội dung thì giữ version cũ, nội dung đổi thì thêm một dòng version + 1, không ghi đè. Hai là quyết định duyệt, lần chạy, đề xuất sửa và sự kiện giao diện chỉ được thêm, không có API sửa hay xoá. Ba là nội dung có cấu trúc lưu dạng JSON để dùng lại đúng model Pydantic của agent, không phải ánh xạ từng trường sang cột.

Bảng 3.7 Các bảng trong cơ sở dữ liệu

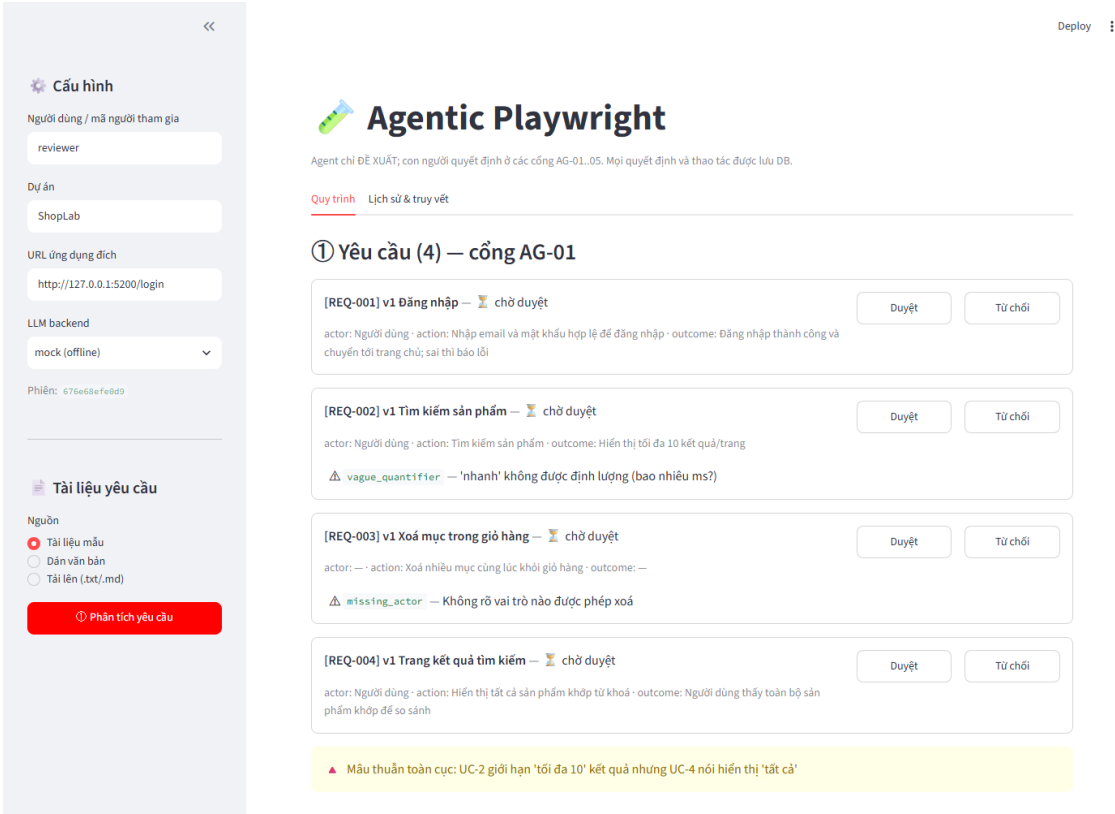
Bảng	Nội dung	Kiểu ghi
project	Dự án và URL ứng dụng đích	tạo một lần
requirement_version	Yêu cầu đã chuẩn hoá (JSON StructuredRequirement)	có version
test_case_version	Test case (JSON TestCase), gắn requirement_id	có version
approval_decision	Cổng, artifact, version, quyết định, người duyệt, lý do	append-only
generated_script	Plan (JSON), mã Python, kết quả grounding	có version
execution_run	Trạng thái, mã thoát, stderr, đường dẫn ảnh chụp	append-only
repair_proposal	Đề xuất sửa (rủi ro, diff, loại thay đổi, plan mới)	append-only
trace_link	Liên kết nguồn → đích và loại quan hệ	thêm khi lưu artifact
ui_event	Thao tác trên giao diện theo phiên và người dùng (dữ liệu RQ4)	append-only

Repository (src/repositories/store.py) là đường truy cập dữ liệu duy nhất của giao diện: nhận và trả model Pydantic, giấu bảng SQLAlchemy phía sau. Trace link được tạo tự động khi lưu artifact: requirement verified_by test_case, test_case implemented_by script@vN, test_case executed_as execution, execution repaired_by repair. Một vòng sửa lỗi vì vậy để lại đủ chuỗi để truy ngược từ đề xuất sửa về yêu cầu gốc.

3.10 Giao diện người dùng

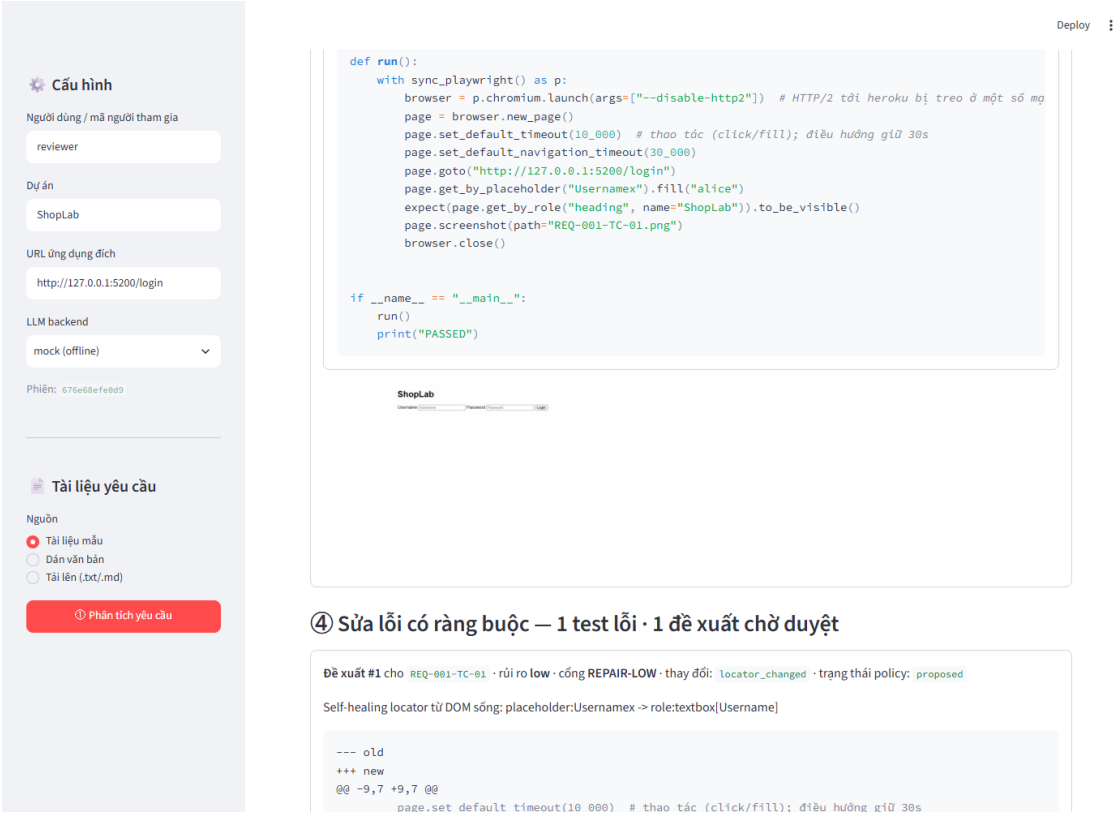
Giao diện Streamlit (src/ui/app.py) đi theo đúng luồng của Hình 3.2 và ghi mọi quyết định vào DB. Thanh bên chọn người dùng, dự án, URL ứng dụng đích, backend LLM và nguồn tài liệu. Tab Quy trình có bốn bước: ① yêu cầu và cổng AG-01 (Hình 3.3); ② test case, sửa được thành version mới, và cổng AG-02; ③ sinh script và chạy; ④

sửa lỗi có ràng buộc. Tab Lịch sử hiển thị số liệu, chuỗi truy vết, các quyết định và thao tác của phiên (Hình 3.5).



Hình 3.3 Bước ①: yêu cầu trích xuất, cờ mờ hồ và nút duyệt AG-01

Ở bước ③, nút Sinh script chạy đúng quy trình DOM-aware có vòng lặp grounding của mục 3.5.4: chụp aria snapshot trang đích, gọi LLM, đếm phần tử khớp trên trang thật, sinh lại một lần nếu có locator khớp khác 1. Người dùng xem bảng grounding, mã sinh ra và ảnh chụp sau khi chạy, và có thể sửa plan JSON trực tiếp (mỗi lần lưu là một version script mới). Ở bước ④, mỗi test lỗi có nút Đề xuất sửa. Đề xuất hiển thị mức rủi ro, công duyệt tương ứng, loại thay đổi, lý do và diff (Hình 3.4). Đề xuất làm đổi ý nghĩa test có cảnh báo đỏ. Chỉ khi người dùng bấm Duyệt và chạy lại, plan mới mới được lưu thành version script mới và chạy lại.



Hình 3.4 Bước ④: đề xuất sửa từ self-healing, rủi ro Low, chờ người duyệt

Bảng 3.8 Ảnh xạ đề xuất sửa sang cổng duyệt

Loại thay đổi trong đề xuất	Cổng	Ý nghĩa
Bỏ, thêm, đổi loại bước; đổi dữ liệu nhập hoặc URL điều hướng	AG-05	Đổi bước kiểm thử
Assertion đổi giá trị kỳ vọng	AG-04	Đổi oracle / expected result
Assertion giữ giá trị nhưng đổi phần tử được kiểm	AG-03	Đổi assertion
Chỉ đổi locator của thao tác	REPAIR-LOW	Duyệt đơn giản

Cấu hình

Người dùng / mã người tham gia
reviewer

Dự án
ShopLab

URL ứng dụng đích
http://127.0.0.1:5200/login

LLM backend
mock (offline) ▼

Phiên: 676e68efed9

Tài liệu yêu cầu

Nguồn

☒ Tài liệu mẫu

☐ Dẫn văn bản

☐ Tài liệu (.txt/.md)

① Phân tích yêu cầu

Deploy ⓘ

Agentic Playwright

Agent chỉ ĐỀ XUẤT; con người quyết định ở các cổng AG-01..05. Mọi quyết định và thao tác được lưu DB.

Quy trình Lịch sử & truy vết

Yêu cầu	Test case	Lần chạy	Pass (lần chạy mỗi nhất)	Đề xuất sửa
4	2	3	1	1

Chuỗi truy vết

nguồn	id nguồn	quan hệ	đích	id đích
requirement	REQ-001	verified_by	test_case	REQ-001-TC-01
requirement	REQ-001	verified_by	test_case	REQ-001-TC-02
test_case	REQ-001-TC-01	implemented_by	script	REQ-001-TC-01@v1
test_case	REQ-001-TC-01	executed_as	execution	1
test_case	REQ-001-TC-01	implemented_by	script	REQ-001-TC-01@v2
test_case	REQ-001-TC-01	executed_as	execution	2
execution	2	repaired_by	repair	1
test_case	REQ-001-TC-01	implemented_by	script	REQ-001-TC-01@v3
test_case	REQ-001-TC-01	executed_as	execution	3

Quyết định (append-only)

thời điểm	artifact	version	quyết định	người duyệt
2026-10-07 10:36:32	REQ-001		1 approved	reviewer
2026-10-07 10:36:34	REQ-001-TC-01		1 approved	reviewer
2026-10-07 10:36:56	repair-1		1 approved	reviewer

Hình 3.5 Tab Lịch sử: chuỗi truy vết tới script v3 và đề xuất sửa, quyết định append-only

Playwright sync API chạy trong tiến trình Streamlit trên Windows để xung đột event loop. Các tác vụ trình duyệt (chụp snapshot, grounding, dò bước hỏng và tự chữa) vì vậy chạy ở tiến trình con qua src/services/browser_tasks.py; harness RQ3 gọi cùng các hàm này trực tiếp trong tiến trình của nó. Luồng giao diện được kiểm bằng một kịch bản Playwright đi hết các bước: chạy đạt, cố ý làm hỏng plan, chạy trượt, nhận đề xuất sửa, duyệt và chạy lại đạt.

3.11 Lớp trừu tượng LLM

Mọi agent gọi LLM qua giao diện LLMClient với một phương thức structured_completion(system, user, schema). Có hai hiện thực: OpenAILLMClient gọi mọi endpoint tương thích OpenAI (OpenAI, DeepSeek, LM Studio, Ollama) bằng base_url, và MockLLMClient trả đối tượng dựng sẵn cho test. Tham số --profile cloud đọc .env.cloud thay cho .env, nên đổi giữa Gemma chạy local và DeepSeek chỉ cần một cờ dòng lệnh. LM Studio từ chối response_format kiểu json_object, nên hệ thống dùng json_schema, kiểu được cả ba backend chấp nhận.

3.12 Kiểm thử phần mềm

Bộ unit test gồm 103 test chạy offline bằng MockLLMClient và hàm đếm giả, không cần mạng hay trình duyệt. Các phần tất định được test kỹ: repair policy, traceability, self-healing, boundary analysis, locator policy, bộ đánh giá RQ1, repository trên SQLite in-memory, hành vi các mutation của ShopLab (qua Flask test client), ánh xạ công duyệt, chấm điểm SUS và logic duyệt của giao diện. Phần cần trình duyệt thật (grounding đa bước, chạy script, chữa theo lô, luồng giao diện) được kiểm chứng bằng chạy thật: oracle RQ2, bước --check của RQ3 và kịch bản Playwright đi hết luồng giao diện.

Chương 4: MÔI TRƯỜNG THỰC NGHIỆM VÀ PHƯƠNG PHÁP ĐÁNH GIÁ

4.1 Môi trường thực nghiệm

Thực nghiệm chạy trên một máy Windows 11, Python 3.14, Playwright 1.62 với Chromium headless. Backend cloud là DeepSeek (model deepseek-chat) qua API tương thích OpenAI. Backend local là google/gemma-4-12b chạy trên LM Studio với GPU 8 GB. Các website đích đều là site demo công khai dựng cho mục đích luyện kiểm thử tự động.

4.2 ShopLab: ứng dụng đích có mutation kiểm soát được

Các site demo công khai không cho phép thay đổi giao diện hay nghiệp vụ theo ý muốn, nên không dùng được để đo khả năng sửa lỗi. Nhóm xây ShopLab (apps/shoplab, Flask, khoảng 200 dòng) gồm đăng nhập, danh sách sản phẩm, giỏ hàng, checkout và hồ sơ. Mọi id, nhãn, placeholder, chữ trên nút, thông báo và cờ hành vi đọc từ cấu hình của biến thể. Mỗi biến thể là phiên bản gốc v0 cộng một nhóm thay đổi, có nhãn ground truth (Bảng 4.1).

Bảng 4.1 Mười bốn biến thể (mutation) của ShopLab

Biến thể	Lớp	Thay đổi
T1_ids	technical	Đổi id của ô nhập và nút đăng nhập
T2_placeholders	technical	Đổi placeholder ô đăng nhập
T3_labels	technical	Đổi nhãn ô đăng nhập
T4_login_text	technical	Đổi chữ nút Login thành Sign in
T5_testids	technical	Đổi data-testid của nút thêm giỏ và badge
T6_checkout_text	technical	Đổi chữ nút Checkout/Continue
T7_classes	technical	Đổi class và bọc thêm div
T8_save_text	technical	Đổi chữ nút Save và nhãn Display name
S1_error_msg	semantic	Đăng nhập sai hiện thông báo chung chung (mất thông tin lỗi)
S2_total	semantic	Tổng tiền giỏ hàng tính sai (bỏ qua món đầu)
S3_validation	semantic	Bỏ kiểm tra bắt buộc First name khi checkout
S4_redirect	semantic	Đăng nhập thành công chuyển nhầm sang trang hồ sơ
S5_locked	semantic	Tài khoản bị khoá vẫn đăng nhập được
S6_add_noop	semantic	Nút Add to cart không thêm gì vào giỏ

Lớp technical là refactor giao diện, nghiệp vụ giữ nguyên: hành vi đúng của cơ chế sửa là tìm lại phần tử, test pass, assertion giữ nguyên. Lớp semantic là thay đổi hành vi

ng nghiệp vụ, tương đương một bug thật: test fail là đúng, và hành vi đúng của cơ chế sửa là không được làm test pass. Bộ test gồm 20 test viết cho v0, cố ý dùng trộn các chiến lược locator (css id, placeholder, label, role, test id, class) như một bộ test thực tế, để mỗi mutation làm gãy một tập test khác nhau. Lệnh `run_rq3.py --check` kiểm rằng 20/20 test pass trên v0 và mỗi mutation làm gãy ít nhất một test.

Trong lần kiểm đầu, mutation đổi placeholder "Username" thành "Enter your username" không làm gãy test nào: `get_by_placeholder` khớp theo chuỗi con, không phân biệt hoa thường. Đây là một tính năng chịu lỗi có thật của Playwright. Biến thể được đổi sang placeholder không chứa từ cũ ("Your login").

4.3 Đánh giá RQ1: yêu cầu có cấu trúc

4.3.1 Dataset

Ground truth được tạo bằng cách chèn khuyết tật có chủ đích vào tài liệu yêu cầu. Tập ban đầu gồm 5 tài liệu (auth, booking, cart, profile, search) với 15 yêu cầu. Tập mở rộng thêm 10 tài liệu thuộc các nghiệp vụ khác (thư viện, chuyển tiền, đăng ký học phần, kho, đặt bàn, ví điện tử, nghỉ phép, mạng xã hội nội bộ, mã giảm giá, hỗ trợ khách hàng), tổng 15 tài liệu và 47 yêu cầu. Khoảng một nửa yêu cầu được chèn một khuyết tật thuộc rubric, hai tài liệu có mâu thuẫn toàn cục; các yêu cầu sạch thường có ràng buộc số để kiểm khả năng sinh ca biên. Mỗi tài liệu có thêm danh sách điều kiện kiểm thử gold (tổng 83 điều kiện: 41 positive, 18 negative, 24 biên), là các hành vi một kiểm thử viên rút được từ văn bản đúng như đang viết. 10 tài liệu mới và toàn bộ điều kiện gold do Claude soạn nháp, nhóm cần review trước khi dùng làm ground truth chính thức.

4.3.2 Phát hiện mơ hồ

Đơn vị so khớp là cặp (yêu cầu, loại mơ hồ); yêu cầu dự đoán được căn với yêu cầu gốc theo thứ tự use case; mâu thuẫn được tính như một loại ở mức toàn cục. Yêu cầu sạch dùng để đo over-flag. Chạy: `python evaluate_agent.py` (thêm `--profile cloud` cho DeepSeek).

4.3.3 Chất lượng test case: hai nhánh

Đây là phần trả lời trực tiếp câu hỏi RQ1. Nhánh direct đưa cả tài liệu cho LLM và yêu cầu sinh test case. Nhánh pipeline chạy Requirement Agent, duyệt mô phỏng mọi yêu

cầu, rồi chạy Test Design Agent và BVA trên từng yêu cầu. Hai nhánh dùng cùng model và cùng các quy tắc chống bịa trong prompt. Một LLM giám khảo nhận tài liệu, danh sách điều kiện gold và các test case, rồi với mỗi test case trả về các điều kiện nó thực sự kiểm và việc kết quả mong đợi của nó có căn cứ trong tài liệu hay không. Từ đó tính độ phủ (tỉ lệ điều kiện gold có ít nhất một test kiểm), tỉ lệ test không căn cứ và tỉ lệ test truy vết được về một yêu cầu có ID. Một mẫu ngẫu nhiên 20% phán quyết được xuất ra CSV để người chấm lại và tính hệ số kappa (python evaluate_rq1.py --kappa). Lệnh chạy: python evaluate_rq1.py --profile cloud.

4.4 Đánh giá RQ2: ngữ cảnh DOM và first-run pass rate

4.4.1 Catalog target

Catalog datasets/reference/rq2_targets.json gồm 50 target trên 6 website (Bảng 4.2), 37 positive và 13 negative. Mỗi target là một test case viết bằng tiếng Việt (các bước và expected_result) trên một URL thật. expected_result mô tả kết quả thật của trang để LLM có căn cứ sinh assertion; RQ2 đo khả năng chuyển test case đã duyệt thành mã, không đo khả năng đoán oracle. Danh sách đầy đủ ở Phụ lục A.

Bảng 4.2 Các website trong catalog RQ2

Website	Số target	Loại kịch bản
The Internet (Heroku)	16	Đăng nhập, JS alert/confirm, nội dung động, redirect, status code
SauceDemo	14	Đăng nhập, giỏ hàng, checkout, chi tiết sản phẩm, đăng xuất
UI Testing Playground	8	Form đăng nhập, đổi tên nút, đếm click, điều hướng
Practice Test Automation	5	Đăng nhập, đăng xuất, thêm dòng động
DemoQA	4	Form text box, radio button, nút bấm
OrangeHRM Demo	3	Đăng nhập và validation

4.4.2 Oracle viết tay: kiểm chứng target khả thi

Một target có thể thất bại vì lý do không liên quan tới LLM: expected_result viết sai, site thay đổi, hoặc kịch bản cần thao tác ngoài DSL (chọn dropdown, hover, nhấn phím). Để loại các nguồn nhiễu này, mỗi target có một trường oracle là plan viết tay bằng đúng DSL mà LLM được dùng. Lệnh python run_rq2.py --context oracle chạy các plan này qua cùng template và Execution Agent.

Trong lúc xây catalog, oracle đã loại nhiều ứng viên. TodoMVC bị loại vì thêm việc cần nhấn Enter. Các trang AJAX và client delay của UI Testing Playground bị loại vì

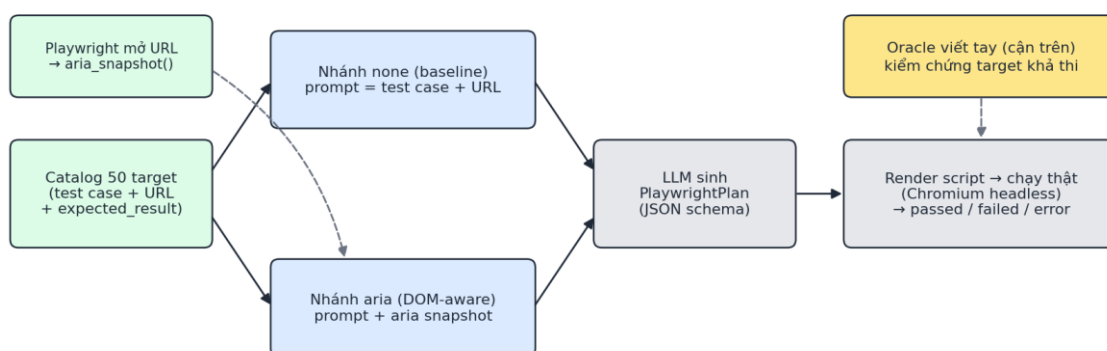
kết quả hiện sau khoảng 15 giây. Dropdown, hover, key press và double click bị loại vì DSL không có thao tác tương ứng. Oracle cũng bắt được lỗi trong catalog cũ: OrangeHRM hiển thị chữ "Required" dưới cả hai ô khi để trống cả hai, nên assertion text khớp hai phần tử và vi phạm strict mode. Target này được sửa thành bỏ trống riêng ô mật khẩu.

4.4.3 Các nhánh so sánh

Cả hai nhánh dùng cùng model, cùng system prompt, cùng 50 test case và cùng template thực thi. Chỉ khác ở ngữ cảnh đưa thêm vào prompt (Hình 4.1):

- none (baseline): prompt gồm URL đích, tiêu đề, loại, các bước và expected_result của test case.
- aria (DOM-aware): prompt như baseline, thêm aria snapshot của trang đích chụp ngay trước khi gọi LLM.
- aria_loop (bản 2): như aria, thêm vòng lặp grounding của mục 3.5.4, tức sinh lại một lần nếu có locator khớp khác 1.

Lần đo đầu (mục 5.2.1–5.2.2) chỉ có hai nhánh none và aria, chạy một lần. Bản 2 chạy cả ba nhánh, lặp ba lần để ước lượng độ dao động giữa các lần sinh của LLM, với template mới (timeout thao tác 10 giây, trường nth). Với template này, oracle đạt 49/50: PT-04 (bấm Log out trên Practice Test Automation) vượt timeout 10 giây vì trang điều hướng chậm. Giới hạn này áp dụng như nhau cho mọi nhánh.



Hình 4.1 Quy trình đo RQ2 trên hai nhánh

Script sinh ra được chạy đúng một lần, không qua Repair Agent hay sửa tay. Plan không hợp lệ (LLM trả JSON hỏng) được tính là error. Mỗi kết quả lưu kèm dòng lỗi

cuối của stderr để phân loại nguyên nhân. Lệnh chạy: `python run_rq2.py --profile cloud --context both --out rq2_results_both.json`.

4.4.4 Kiểm soát nhiều hạ tầng

Khi chạy oracle lần đầu, mọi target của The Internet đều fail ở page.goto do timeout 30 giây, trong khi curl tải đúng các tài nguyên đó dưới 1 giây. Theo dõi request cho thấy Chromium kẹt khi tải tệp js và png tĩnh qua HTTP/2 từ mạng của nhóm. Thêm cờ `--disable-http2` thì cả ba lần thử đều tải xong trong 3,3 giây. Lỗi này cũng có thể đã làm giảm kết quả của lần đo thử ngày 15/09 (8/19 target đạt), trong đó The Internet chỉ đạt 3/7.

Sau khi tắt HTTP/2, The Internet vẫn thỉnh thoảng treo khi tải trang. Đây là lỗi hạ tầng, không phải lỗi script, nên khi một script fail với thông điệp Page.goto: Timeout, hệ thống chạy lại đúng script đó một lần và đánh dấu `infra_retry` trong JSON. Script không đổi nên phép đo first-run vẫn giữ nguyên ý nghĩa: LLM chỉ có một cơ hội sinh mã.

4.5 Đánh giá RQ3: sửa lỗi có ràng buộc

Mỗi cặp (test, mutation) mà test gây là một ca sửa lỗi. Để mutation làm gây nhiều test (T4 làm gây 17, S4 làm gây 12) không lẫn át số liệu, mỗi mutation lấy tối đa 6 test gây đầu tiên theo thứ tự cố định. Mỗi ca được chạy qua ba nhánh (Bảng 4.3), cùng ngân sách 2 lần sửa và cùng bằng chứng: stderr và aria snapshot của trang tại bước hỏng.

Bảng 4.3 Ba nhánh sửa lỗi của RQ3

Nhánh	Cách sửa	Áp dụng đề xuất
unconstrained	LLM với prompt "làm test PASS, được đổi mọi action kể cả assertion" (mô phỏng autonomous repair [8])	Tự áp dụng mọi đề xuất
heal	Chỉ self-healing tắt định từ DOM sống, không LLM	Áp dụng (luôn là rủi ro Low)
constrained	Hệ thống: self-healing trước, LLM có ràng buộc sau; repair_policy phân mức	Người duyệt mô phỏng duyệt Low; Medium, High và blocked chuyển lên người

Số ca theo mutation: T1_ids 3; T2_placeholders 6; T3_labels 4; T4_login_text 6; T5_testids 6; T6_checkout_text 2; T7_classes 3; T8_save_text 1; S1_error_msg 2; S2_total 2; S3_validation 1; S4_redirect 6; S5_locked 1; S6_add_noop 5.

Số đo: số mutation technical sửa được mà không làm yếu assertion; số mutation semantic bị che (sau sửa test pass dù app đang lỗi); số lần làm yếu assertion (đề xuất được áp dụng có đổi assertion, đổi loại action hoặc bỏ bước); số ca escalate lên người. Người duyệt mô phỏng là một giả định: trong thực tế người duyệt có thể duyệt cả đề xuất High (nếu yêu cầu thật sự đổi) hoặc từ chối cả đề xuất Low.

4.6 Đánh giá RQ4: thí nghiệm có người tham gia

RQ4 cần đo công sức của người dùng, nên không mô phỏng được. Giao thức đầy đủ ở docs/development/rq4-protocol.md. Thiết kế trong-đối-tượng: mỗi người tham gia làm hai phiên 30 phút, một phiên tự viết test Playwright (được dùng codegen, không dùng trợ lý AI), một phiên dùng giao diện của hệ thống, trên hai bộ nhiệm vụ tương đương A và B (mỗi bộ 3 yêu cầu trên ShopLab). Thứ tự điều kiện và bộ nhiệm vụ được đảo cân bằng giữa người tham gia.

Bảng 4.4 Số đo RQ4

Số đo	Cách lấy
Thời gian hoàn thành	Bấm giờ bằng tools/rq4/rq4.py timer
Số yêu cầu có test hợp lệ (0–3)	Tự chấm: test phải pass trên ShopLab v0 và fail trên mutation semantic phá đúng hành vi của yêu cầu đó
Số lần sửa tay (điều kiện hệ thống)	Sự kiện edit_test_case, edit_plan trong bảng ui_event
Mức dễ dùng	Phiếu SUS 10 câu (0–100)

Tiêu chí test hợp lệ bắt được test chạy được nhưng không kiểm gì. Khi chạy thử bộ chấm, một test thêm sản phẩm vào giỏ mà không kiểm số trên giỏ được chấm không hợp lệ, vì nó vẫn pass trên mutation làm nút Add to cart vô hiệu. Hai điều kiện được chấm bằng cùng một bộ chấm.

Chương 5: KẾT QUẢ THỰC NGHIỆM VÀ ĐÁNH GIÁ

5.1 RQ1: yêu cầu có cấu trúc

5.1.1 Phát hiện mơ hồ trên tập ban đầu

Bảng 5.1 là kết quả trên 15 yêu cầu sau khi siết prompt. Dòng Gemma "chưa nắn prompt" là cấu hình trước khi thêm chỉ dẫn gắn cờ theo từng yêu cầu (mục 3.3).

Bảng 5.1 Kết quả phát hiện mơ hồ trên tập 15 yêu cầu

Model	Micro-P	Micro-R	Micro-F1	Macro-F1	Over-flag
deepseek-chat (cloud)	0,50	1,00	0,67	0,71	0,20
gemma-4-12b, chưa nắn prompt	0,06	0,08	0,07	0,17	0,00
gemma-4-12b (local)	0,71	0,83	0,77	0,69	0,00

Siết prompt (chỉ gắn cờ khi thiếu hoặc mơ hồ rõ ràng, ưu tiên không gắn khi phân vân ở precondition và outcome) giảm over-flag ở cả hai backend (Bảng 5.2). Đổi lại, recall của Gemma giảm từ 1,00 xuống 0,83, chủ yếu do bỏ sót missing_precondition; DeepSeek giữ recall 1,00.

Bảng 5.2 Ảnh hưởng của việc siết prompt (trước → sau)

Model	Micro-F1	Over-flag	Recall
gemma-4-12b (local)	0,73 → 0,77	0,40 → 0,00	1,00 → 0,83
deepseek-chat (cloud)	0,51 → 0,67	0,80 → 0,20	1,00 → 1,00

Bảng 5.3 F1 theo từng loại mơ hồ

Loại	deepseek-chat	gemma-4-12b
conflict	1,00	1,00
missing_actor	0,86	1,00
missing_expected_outcome	0,60	0,67
missing_precondition	0,50	0,00
underspecified_action	0,29	0,50
vague_quantifier	1,00	1,00

Dưới cùng một prompt, Gemma có precision và micro-F1 cao hơn, DeepSeek bắt đủ hơn (recall và macro-F1 cao hơn). missing_precondition là loại khó nhất với cả hai. Nhân gold của loại này cũng chủ quan nhất: mỗi yêu cầu chỉ có một khuyết tật được chèn, trong khi một người đọc khác có thể thấy thêm precondition còn thiếu. Với 15

yêu cầu, các con số này chỉ là sơ bộ; một yêu cầu đôi nhãn có thể làm F1 của một loại thay đổi vài chục điểm phần trăm.

5.1.2 Phát hiện mơ hồ trên tập mở rộng

Tập mở rộng gồm 15 tài liệu, 47 yêu cầu, trong đó 23 yêu cầu sạch. Với deepseek-chat: micro-precision 0,44, micro-recall 0,90, micro-F1 0,59, macro-F1 0,60; 5/23 yêu cầu sạch bị gắn cờ nhầm.

Bảng 5.4 Phát hiện mơ hồ theo loại trên 47 yêu cầu (deepseek-chat)

Loại	TP/FP/FN	Precision	Recall	F1
conflict	3/0/0	1,00	1,00	1,00
missing_actor	6/6/0	0,50	1,00	0,67
missing_expected_outcome	9/10/2	0,47	0,82	0,60
missing_precondition	1/9/0	0,10	1,00	0,18
underspecified_action	1/11/1	0,08	0,50	0,14
vague_quantifier	8/0/0	1,00	1,00	1,00

Trên tập mở rộng, recall vẫn cao (0,90) nhưng precision giảm còn 0,44. Hai loại kéo precision xuống là missing_precondition (1 đúng, 9 gắn nhầm) và underspecified_action (1 đúng, 11 gắn nhầm). Nhiều yêu cầu trong tập không nêu điều kiện trước, ví dụ đăng ký học phần hay xuất kho không nói người thao tác phải đăng nhập, nên gắn cờ missing_precondition cho chúng không hẳn là sai. Nhãn gold chỉ chèn một khuyết tật chủ đích cho mỗi yêu cầu, nên các khuyết tật tự nhiên khác của văn bản bị tính là FP. Vì vậy precision đo được là cận dưới; để kết luận chắc hơn cần nhiều người gán nhãn độc lập trên cùng tập.

conflict và vague_quantifier đạt F1 1,00 trên cả tập 15 và tập 47 yêu cầu, cả ba mâu thuẫn toàn cục đều được bắt.

5.1.3 Chất lượng test case: pipeline có cấu trúc so với đưa thẳng tài liệu

Bảng dưới so sánh hai nhánh trên 15 tài liệu và 83 điều kiện gold, qua hai phiên bản quy tắc sinh test. Phiên bản 1 dùng prompt ban đầu của Test Design Agent. Phiên bản 2 thêm quy tắc chỉ sinh negative, error_guessing và alternative_flow khi tài liệu nêu phản ứng của hệ thống, áp dụng cho cả hai nhánh; ca biên BVA mang theo hành động của yêu cầu. Dòng "có AG-01" chỉ tính test sinh từ yêu cầu không bị Requirement Agent gắn cờ mơ hồ, mô phỏng việc người duyệt giữ các yêu cầu mơ hồ lại để làm rõ.

Bảng 5.5 Chất lượng test case theo nhánh và phiên bản quy tắc

Nhánh	Số TC	Độ phủ gold	Positive	Negative	Biên	Không căn cứ	Truy vết
direct v1	168	79/83 (95%)	41/41	15/18	23/24	32%	0%
pipeline v1	304	76/83 (92%)	41/41	13/18	22/24	45%	100%
direct v2	79	62/83 (75%)	38/41	8/18	16/24	3%	0%
pipeline v2	106	67/83 (81%)	41/41	8/18	18/24	9%	100%
pipeline v2 có AG-01	72	43/83 (52%)	21/41	4/18	18/24	4%	100%

So sánh theo cặp trên từng điều kiện gold (McNemar chính xác): phiên bản 1: 1 điều kiện chỉ pipeline phủ, 4 chỉ direct phủ, $p = 0,38$; phiên bản 2: 8 điều kiện chỉ pipeline phủ, 3 chỉ direct phủ, $p = 0,23$.

Bảng 5.6 Test không căn cứ theo loại test (phiên bản 1)

Loại test	Direct: không căn cứ / tổng	Pipeline: không căn cứ / tổng
alternative_flow	4/12	13/27
boundary	8/37	9/38
error_guessing	30/36	52/64
negative	11/42	56/114
positive	0/41	8/61

Ở phiên bản 1, đưa thẳng tài liệu cho LLM cho độ phủ 95% với 168 test, còn pipeline cho 92% với 304 test. Chênh lệch độ phủ không có ý nghĩa thống kê. Điểm khác lớn là tỉ lệ test không căn cứ: 32% ở direct và 45% ở pipeline. Bảng theo loại test cho thấy nguồn gốc: error_guessing gần như luôn bị bịa ở cả hai nhánh, và pipeline sinh nhiều negative hơn hẳn (114 so với 42) vì Test Design Agent làm việc trên từng yêu cầu một và cố tìm ca âm bản cho mỗi yêu cầu kể cả khi tài liệu không nói hệ thống phản ứng thế nào. Ví dụ ở tài liệu thư viện, pipeline sinh "gia hạn sách đã trả thì bị từ chối", một luật tài liệu không hề nêu.

Tách theo use case, pipeline có 34% test không căn cứ ở use case sạch và 58% ở use case có khuyết tật chèn sẵn. Nghĩa là phần lớn test bịa xuất phát từ yêu cầu mơ hồ, đúng loại yêu cầu mà công AG-01 được thiết kế để chặn.

Phiên bản 2 thêm một quy tắc cho cả hai nhánh. Tỉ lệ không căn cứ giảm mạnh (direct 32% xuống 3%, pipeline 45% xuống 9%) nhưng độ phủ cũng giảm (direct 95% xuống 75%, pipeline 92% xuống 81%), chủ yếu ở điều kiện negative và biên: model trở nên dè dặt cả với ca tài liệu có nêu. Dưới cùng quy tắc, pipeline phủ nhiều hơn direct 5

điều kiện, rõ nhất ở ca biên nhờ BVA tắt định, nhưng chênh lệch vẫn không có ý nghĩa thống kê. Khi mô phỏng thêm cổng AG-01, test không căn cứ còn 4% nhưng độ phủ chỉ còn 52%: các yêu cầu mơ hồ chưa được làm rõ thì chưa có test.

Trả lời RQ1 trên tập này: chuẩn hoá yêu cầu có cấu trúc không làm tăng độ phủ có ý nghĩa thống kê so với đưa thẳng tài liệu. Lợi ích đo được nằm ở chỗ khác. Mọi test truy vết được về một yêu cầu có ID (100% so với 0%). Cờ mơ hồ cho phép chặn đúng phần yêu cầu sinh ra nhiều test bịa nhất. Ca biên được bảo đảm bằng BVA. Yếu tố ảnh hưởng mạnh nhất tới độ tin cậy của test là quy tắc sinh test: đổi một quy tắc làm tỉ lệ test bịa giảm từ 32–45% xuống 3–9%, mạnh hơn nhiều so với khác biệt giữa hai nhánh.

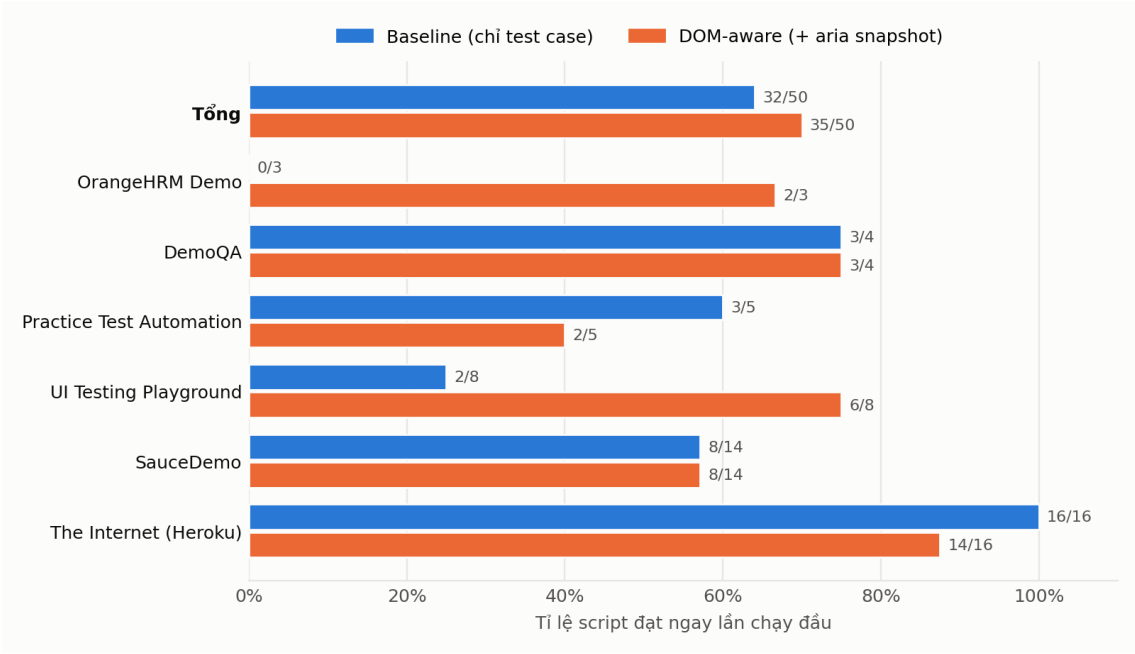
Giám khảo là chính model đã sinh test (DeepSeek), có thể thiên vị cách diễn đạt của nó. Một mẫu ngẫu nhiên 20% các phán quyết đã được xuất ra CSV để nhóm chấm lại và tính hệ số kappa; khi chưa có kết quả chấm tay, các con số độ phủ và không căn cứ cần được đọc như ước lượng của giám khảo LLM.

5.2 RQ2: ngữ cảnh DOM và first-run pass rate

Oracle viết tay đạt 50/50 target, nên catalog khả thi với DSL hiện có và đóng vai cận trên của phép đo. Với model deepseek-chat, nhánh baseline đạt 32/50 (64%), nhánh có aria snapshot đạt 35/50 (70%).

Bảng 5.7 First-run pass rate theo website, hai nhánh

Website	Số target	Baseline (none)	DOM-aware (aria)
The Internet (Heroku)	16	16 (100%)	14 (88%)
SauceDemo	14	8 (57%)	8 (57%)
UI Testing Playground	8	2 (25%)	6 (75%)
Practice Test Automation	5	3 (60%)	2 (40%)
DemoQA	4	3 (75%)	3 (75%)
OrangeHRM Demo	3	0 (0%)	2 (67%)
Tổng	50	32 (64%)	35 (70%)



Hình 5.1 Tỷ lệ đạt lần đầu theo website, baseline và DOM-aware

Hai nhánh chạy trên cùng target nên có thể so sánh từng cặp (bảng dưới).

Bảng 5.8 Bảng chéo kết quả theo cặp target

	aria đạt	aria trượt
none đạt	29	3
none trượt	6	12

Có 6 target chỉ nhánh aria đạt và 3 target chỉ baseline đạt, Kiểm định McNemar chính xác trên 9 cặp bất đồng cho $p = 0,508$, không nhỏ hơn 0,05, nên với cỡ mẫu 50 chưa đủ bằng chứng thống kê để khẳng định khác biệt.

Bảng 5.9 Nguyên nhân thất bại theo nhánh

Nguyên nhân (từ dòng lỗi cuối của stderr)	Baseline	DOM-aware
Không tìm thấy phần tử để thao tác (timeout)	13	5
Locator khớp nhiều phần tử (strict mode)	5	9
Site không tải được (hạ tầng)	0	1
Tổng số target trượt	18	15

Trong 100 lần chạy có 1 lần cần chạy lại do Page.goto: Timeout (mục 4.4.4).

5.2.1 Lần đo thứ nhất: một lỗi đặc tả prompt bị che khuất

Lần đo đầu tiên cho baseline 21/50 (42%) và aria 38/50 (76%). Đọc từng script lỗi thì thấy 17 script baseline bắt đầu bằng `page.goto("")`, trong khi nhánh aria chỉ có 1. Gọi lại LLM trên ba target cho thấy DeepSeek đều đặt URL vào trường value thay vì arg. Nguyên nhân nằm ở system prompt: phần mô tả trường có nói fill dùng arg cho

giá trị nhập và expect_url dùng arg cho URL, nhưng bỏ sót goto. Khi prompt dài thêm vì có aria snapshot, model lại điền đúng.

Nếu dừng ở lần đo này, khoảng cách giữa hai nhánh sẽ bị quy hết cho ngữ cảnh DOM, trong khi phần lớn nó đến từ một lỗi đặc tả không liên quan tới locator. Nhóm thêm một dòng "goto: arg = URL" vào prompt (áp dụng cho cả hai nhánh) rồi đo lại toàn bộ. Các kết quả đầu mục 5.2 là của lần đo thứ hai; bảng dưới đặt hai lần đo cạnh nhau.

Bảng 5.10 Hai lần đo RQ2 trước và sau khi sửa đặc tả goto

Lần đo	Baseline đạt	Aria đạt	goto rỗng (baseline / aria)
1 (prompt thiếu đặc tả goto)	21/50	38/50	17 / 1
2 (đã sửa prompt)	32/50	35/50	0 / 0

5.2.2 Thảo luận về lần đo đầu

Sau khi sửa đặc tả goto, nhánh aria đạt 35/50 so với 32/50 của baseline, hơn 3 target. Có 6 target chỉ aria đạt và 3 target chỉ baseline đạt, McNemar cho $p \approx 0,51$. Với 50 target và một lần chạy, nhóm chưa có bằng chứng thống kê rằng aria snapshot của trang đầu cải thiện first-run pass rate tổng thể.

Phân tích theo nguyên nhân cho thấy rõ hơn ngữ cảnh DOM đã làm gì. Nhóm lỗi "không tìm thấy phần tử để thao tác" giảm từ 13 xuống 5. Ví dụ điển hình là UI Testing Playground (2/8 lên 6/8) và OrangeHRM (0/3 lên 2/3). Không có snapshot, LLM đoán get_by_label("Password"), nhưng ô mật khẩu của Sample App không có nhãn, chỉ có placeholder "*****", còn nhãn của OrangeHRM không gắn với ô nhập. Snapshot cho LLM thấy tên truy cập thật (textbox "*****", textbox "Username") nên nó viết get_by_role với đúng tên đó. Đây chính là loại lỗi mà grounding nhắm tới: locator trỏ tới một phần tử không tồn tại.

Nhóm lỗi "locator khớp nhiều phần tử" lại tăng từ 5 lên 9. Khi có snapshot, LLM dùng đúng tên nó thấy, ví dụ button "Add to cart", trong khi trang sản phẩm của SauceDemo có sáu nút cùng tên. Practice Test Automation in nguyên văn thông báo lỗi trong phần hướng dẫn trên trang, nên get_by_text("Your username is invalid!") khớp hai phần tử. Snapshot cho biết phần tử có tồn tại nhưng không cho biết locator có duy nhất hay không. DSL hiện tại cũng chưa có cách thu hẹp phạm vi (lọc theo khung sản phẩm, chọn phần tử thứ n). Ngữ cảnh DOM vì thế đổi loại lỗi từ "không tồn tại" sang "không

duy nhất". Đây đúng là chỗ phân biệt locator validity với tính duy nhất trong GAP-03. Bước grounding đếm số phần tử khớp (mục 3.5.2) phát hiện được loại lỗi này trước khi chạy, nên hướng hợp lý là ghép hai cơ chế: snapshot cho LLM, grounding kiểm tính duy nhất, rồi trả kết quả đếm về cho LLM sinh lại.

Snapshot chỉ chụp trang đầu nên không giúp với phần tử xuất hiện sau điều hướng. Ở TI-15 và PT-04, nút đăng xuất chỉ có sau khi đăng nhập. Baseline đoán role link và đặt; nhánh aria đoán role button và trượt. Ba target chỉ baseline đạt đều rơi vào trường hợp này hoặc lỗi hạ tầng: TI-08 của nhánh aria trượt vì The Internet không tải được cả sau lần chạy lại. Snapshot theo từng bước, dựa trên `ground_flow`, là cải tiến trực tiếp cho điểm yếu này.

Bài học phương pháp từ lần đo thứ nhất đáng ghi lại. Lần đó cho 42% so với 76%, với 19 target chỉ aria đạt và 2 target chỉ baseline đạt ($p < 0,001$). Nhìn con số tổng, kết luận "ngữ cảnh DOM gần như gấp đôi tỉ lệ đạt" rất hấp dẫn, nhưng nó sai: phần lớn khoảng cách đến từ một dòng thiếu trong prompt. Hai việc giúp bắt được lỗi này là đọc từng script trượt thay vì chỉ đọc tỉ lệ, và có oracle chứng minh mọi target đều làm được, nên lỗi phải nằm ở phía sinh mã.

5.2.3 Bản 2: grounding vòng lặp, ba lần lặp

Bảng 5.11 First-run pass rate qua 3 lần lặp (50 target, deepseek-chat)

Nhánh	Số target đạt từng lần	Trung bình	Tỉ lệ TB
none	34 / 34 / 36	$34,7 \pm 1,2$	69%
aria	39 / 41 / 41	$40,3 \pm 1,2$	81%
aria_loop	49 / 48 / 50	$49,0 \pm 1,0$	98%

Bảng 5.12 So sánh theo cặp từng lần lặp

Lần	Cặp	Chỉ nhánh sau đạt	Chỉ nhánh trước đạt	p (McNemar)
1	none → aria	9	4	0,267
1	none → aria_loop	16	1	< 0,001
1	aria → aria_loop	10	0	0,002
2	none → aria	10	3	0,092
2	none → aria_loop	16	2	0,001
2	aria → aria_loop	7	0	0,016
3	none → aria	7	2	0,180
3	none → aria_loop	14	0	< 0,001
3	aria → aria_loop	9	0	0,004

Số target mà vòng lặp phải sinh lại (có locator khớp khác 1 khi grounding) ở mỗi lần: 10, 9, 8 trên 50.

Oracle viết tay cũng rỗng ở 7 target (PT-02, PT-04, PT-05, SD-13, TI-04, UP-03, UP-08): trạng thái kỳ vọng đã có sẵn trước khi thao tác, nên bất kỳ test nào kiểm bằng văn bản hay URL cũng pass. Bảng dưới tách hai loại: rỗng do target (cũng rỗng ở oracle) và rỗng do LLM (oracle không rỗng, script LLM thì rỗng).

Bảng 5.13 Kiểm tra assertion rỗng trên các script đã pass

Lần	Nhánh	Pass	Kiểm được	Rỗng do target	Rỗng do LLM
1	aria	39	36	1	—
1	aria_loop	49	45	4	PT-03, UP-05
2	none	34	31	4	—
2	aria	41	38	2	—
2	aria_loop	48	44	4	PT-03, UP-05
3	none	36	33	4	—
3	aria	41	38	3	—
3	aria_loop	50	46	5	PT-03, UP-05

Ba lần lặp cho kết quả ổn định. Nhánh aria đạt 39–41/50, cao hơn baseline 34–36/50, nhưng ở cả ba lần chênh lệch đều không có ý nghĩa thống kê (p từ 0,09 đến 0,27). Nhánh aria_loop đạt 48–50/50, hơn baseline và hơn aria có ý nghĩa thống kê ở cả ba lần ($p \leq 0,016$), và không làm mất target nào mà aria đạt. Vòng lặp phải sinh lại 8–10 target mỗi lần. Như vậy, đưa cây trợ năng vào prompt là chưa đủ: phần tạo ra khác biệt là phản hồi grounding, tức đếm phần tử khớp trên trang thật kể cả sau điều hướng, rồi cho LLM sửa đúng chỗ locator khớp 0 hoặc nhiều phần tử.

Tỉ lệ pass gần 100% buộc phải hỏi thêm: các test pass này có thật sự kiểm hành vi không. Phép kiểm assertion rỗng (tools/rq2_vacuity.py) bỏ mọi bước fill và click khỏi script đã pass rồi chạy lại; vẫn pass nghĩa là assertion không phụ thuộc vào hành động. Chạy phép kiểm trên chính oracle viết tay cho thấy một số target yếu từ gốc. Ở SD-13 và PT-04 (đăng xuất rồi kiểm đang ở trang đăng nhập) và UP-03, trạng thái kỳ vọng trùng với trạng thái ban đầu. Ở PT-02, thẻ #error chứa sẵn câu thông báo trong DOM ngay khi tải trang và chỉ được hiện ra bằng CSS, nên mọi assertion bằng văn bản đều pass mà không cần bấm Submit. Catalog cần bổ sung assertion phân biệt được trạng thái trước và sau (ví dụ kiểm class hiển thị), nằm ngoài DSL hiện tại.

Trừ các target yếu, phép kiểm tìm ra assertion rỗng do chính LLM gây ra, và chúng tập trung ở nhánh aria_loop. Ở cả ba lần, PT-03 dùng get_by_text("Your password is invalid!").nth(0) và nth(0) trở vào đoạn hướng dẫn trên trang chứ không phải thông báo lỗi. UP-05 không có bước thao tác nên phép kiểm tự động không áp dụng; kiểm tay cho thấy get_by_text("Welcome UserName!").nth(0) trở vào một thẻ code trong đoạn giải thích. Cả hai đều xuất phát từ cùng một cơ chế: khi grounding báo locator khớp nhiều phần tử, LLM chọn cách rẻ nhất là thêm nth để locator khớp đúng một, và grounding chỉ đếm số lượng nên chấp nhận. Locator hợp lệ về kỹ thuật nhưng sai về ngữ nghĩa, đúng loại rủi ro mà GAP-03 nêu. Sau khi trừ 2 ca này, aria_loop còn 46–48 pass có ý nghĩa mỗi lần, vẫn cao hơn rõ rệt aria (39–41) và baseline (33–35, ở hai lần lặp còn script để kiểm).

Trả lời RQ2: kết hợp yêu cầu với cây trợ năng cải thiện tỉ lệ script chạy đạt ở lần đầu khi cây trợ năng được dùng trong một vòng kiểm chứng (68–72% lên 96–100%), còn chỉ đưa vào prompt thì cải thiện chưa đủ chắc chắn. Grounding bằng cách đếm phần tử không đủ để bảo đảm locator đúng ngữ nghĩa; cần thêm một tầng kiểm tra như phép kiểm assertion rỗng, hoặc cấm nth trên assertion.

5.3 RQ3: sửa lỗi có ràng buộc

Bảng 5.14 Kết quả ba nhánh sửa trên ShopLab (deepseek-chat, ngân sách 2)

Nhánh	Technical sửa được (giữ assertion)	Semantic bị che	Escalate	Lần làm yếu assertion
unconstrained	23/31	8/17	0	18
heal	24/31	0/17	0	0
constrained	24/31	0/17	11	0

Lần chạy đầu, khi self-healing chỉ chữa một locator mỗi lượt: unconstrained sửa 23/31, che 10/17, escalate 0; heal sửa 21/31, che 0/17, escalate 0; constrained sửa 21/31, che 0/17, escalate 11.

Bảng 5.15 Kết quả theo mutation (sửa được / escalate / không sửa được)

Mutation	Lớp	unconstrained	heal	constrained
T1_ids	technical	0/0/3	3/0/0	3/0/0
T2_placeholders	technical	6/0/0	6/0/0	6/0/0
T3_labels	technical	4/0/0	4/0/0	4/0/0
T4_login_text	technical	6/0/0	6/0/0	6/0/0
T5_testids	technical	6/0/0	4/0/2	4/2/0
T6_checkout_text	technical	2/0/0	0/0/2	0/1/1

Mutation	Lớp	unconstrained	heal	constrained
T7_classes	technical	3/0/0	0/0/3	0/3/0
T8_save_text	technical	1/0/0	1/0/0	1/0/0
S1_error_msg	semantic	2/0/0	0/0/2	0/0/2
S2_total	semantic	2/0/0	0/0/2	0/0/2
S3_validation	semantic	0/0/1	0/0/1	0/0/1
S4_redirect	semantic	4/0/2	0/0/6	0/5/1
S5_locked	semantic	0/0/1	0/0/1	0/0/1
S6_add_noop	semantic	0/0/5	0/0/5	0/0/5

Kết quả chính ổn định qua hai lần chạy. Nhánh unconstrained biến 8/17 lỗi nghiệp vụ thật thành test pass (lần đầu 10/17) và làm yếu assertion 18 lần (lần đầu 21). Hai kiểu che lỗi lặp lại đúng như [8] mô tả. Kiểu thứ nhất là sửa giá trị kỳ vọng cho khớp với bug: ở S1, assertion "Invalid username or password" bị đổi thành "Something went wrong"; ở S2, tổng tiền kỳ vọng "Total: \$39.98" bị đổi thành "Total: \$9.99", đúng con số sai mà app đang hiển thị vì bỏ qua món đầu tiên. Kiểu thứ hai là chèn thêm bước: ở S4, app chuyển nhầm sang trang hồ sơ sau đăng nhập, agent thêm một lệnh goto tới trang sản phẩm để test đi tiếp. Mỗi đề xuất đều "hợp lý" nếu nhìn riêng, và đều làm test mất khả năng phát hiện đúng lỗi nó được viết ra để bắt.

Nhánh constrained không che lỗi nào trong 17 ca semantic ở cả hai lần chạy: 5 ca được đẩy lên người duyệt với mức High (đề xuất đổi assertion hoặc thêm bước), 12 ca còn lại vẫn fail vì không có cách sửa mức Low nào làm test pass. Cả hai đều là hành vi đúng. Với mutation technical, nhánh này sửa được 24/31 mà không làm yếu assertion nào, ngang nhánh unconstrained (23/31 sửa được mà không làm yếu, thêm 5 ca sửa được nhưng có làm yếu).

Cái giá là 6/31 ca technical bị escalate oan. Cả sáu đều cần đổi locator bên trong một assertion: T5 đổi test-id của badge giỏ hàng, T7 đổi class của dòng tổng tiền. Policy hiện xếp mọi thay đổi trong assertion vào mức High, nên người duyệt phải xem các đề xuất này dù giá trị kỳ vọng không đổi. Hàm repair_gate đã phân biệt được trường hợp này (AG-03: đổi phần tử được kiểm, khác AG-04: đổi giá trị kỳ vọng), nên một hướng cải tiến là hạ nó xuống Medium kèm bằng chứng grounding. Đổi lại, một assertion trở sang phần tử khác cũng có thể là cách làm yếu test (xem phần assertion rỗng ở RQ2), nên nhóm giữ mức High ở phiên bản này.

Chữa theo lô nâng số mutation technical sửa được từ 21 lên 24 ở cả nhánh heal và constrained: mutation đổi id cả ba ô đăng nhập giờ được chữa trong một đề xuất thay vì cần ba lượt. Nhánh heal chỉ dùng self-healing, không gọi LLM, đạt cùng 24/31 và không che lỗi nào. Trên ShopLab, phần LLM trong nhánh constrained hầu như không đóng góp thêm lần sửa nào được áp dụng: các đề xuất LLM đều chạm vào assertion hoặc bước, nên bị chuyển lên người.

Trả lời RQ3 trên tập này: sửa có ràng buộc giữ được tỉ lệ sửa thành công tương đương sửa tự do với lỗi giao diện (24/31 so với 23/31 không làm yếu assertion), trong khi loại bỏ hoàn toàn việc che lỗi nghiệp vụ (0/17 so với 8–10/17). Chi phí là số đề xuất phải người duyệt: 11 ca mỗi lần chạy.

5.4 RQ4: công sức thủ công

Thí nghiệm có người tham gia chưa được tiến hành ở thời điểm viết chuyên đề. Toàn bộ công cụ đã sẵn sàng: hai bộ nhiệm vụ tương đương, giao diện ghi nhật ký thao tác vào DB, lệnh bấm giờ, xuất script từ DB và bộ chấm tự động bằng mutation. Bộ chấm đã được chạy thử với một bộ test viết tay: test kiểm đăng nhập được chấm hợp lệ, còn test thêm giỏ hàng không kiểm số trên giỏ bị chấm không hợp lệ đúng như thiết kế.

5.5 Môi đe dọa tính hợp lệ

- Ground truth RQ1 (10/15 tài liệu và toàn bộ 83 điều kiện gold) do Claude soạn nháp; nhân mơ hồ chỉ chèn một khuyết tật mỗi yêu cầu. Cần nhóm review và nhiều người gán nhãn độc lập.
- Giám khảo RQ1 là chính model sinh test. Mẫu 20% đã xuất ra để chấm tay, kappa chưa có.
- Các quy tắc sinh test (RQ1 v2), trường nth, vòng lặp grounding và chữa theo lô được thêm SAU khi đọc kết quả lần đầu trên cùng dữ liệu. Các con số sau cải tiến là ước lượng lạc quan; cần tập kiểm tra mới để xác nhận.
- RQ2 dùng site demo, luồng ngắn; ShopLab của RQ3 nhỏ và do nhóm dựng. Kết quả không suy rộng cho ứng dụng doanh nghiệp.
- Thực nghiệm chủ yếu chạy với một model (deepseek-chat). Kết quả có thể khác với model khác.

- Người duyệt trong RQ3 là mô phỏng (duyet mọi đề xuất Low, chuyển phần còn lại lên người).
- Hạ tầng mạng ảnh hưởng tới RQ2 (HTTP/2 tới Heroku); đã kiểm soát bằng --disable-http2 và chạy lại khi treo tải trang.

Chương 6: KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

6.1 Kết quả đạt được

- Xây dựng được hệ thống chạy trọn luồng từ tài liệu yêu cầu tới script Playwright Python trên website thật: năm agent, giao diện Streamlit với các công duyệt AG-01..05, lưu version, quyết định duyệt append-only và chuỗi truy vết trong DB, kèm ứng dụng đích ShopLab và bộ công cụ thực nghiệm cho cả bốn câu hỏi nghiên cứu.
- RQ1: trên 15 tài liệu và 83 điều kiện gold, cấu trúc hoá yêu cầu không làm tăng độ phủ test case có ý nghĩa thống kê so với đưa thẳng tài liệu cho LLM. Lợi ích đo được là truy vết 100% và khả năng chặn yêu cầu mơ hồ ở AG-01, nơi phát sinh phần lớn test bịa. Quy tắc sinh test ảnh hưởng tới tỉ lệ test bịa mạnh hơn nhiều so với việc có cấu trúc.
- RQ2: trên 50 target thật, qua ba lần lặp, vòng lặp grounding (aria snapshot cộng phản hồi số phần tử khớp trên trang thật) nâng tỉ lệ script chạy đạt ở lần đầu từ 68–72% lên 96–100%, có ý nghĩa thống kê ở cả ba lần; chỉ đưa snapshot vào prompt thì chưa đủ chắc chắn. Phép kiểm assertion rộng cho thấy 2 trong số các pass đó là pass giả do nth trả sai phần tử, nên grounding theo số lượng cần thêm kiểm tra ngữ nghĩa.
- RQ3: trên ShopLab, sửa lỗi có ràng buộc sửa được 24/31 lỗi giao diện mà không làm yếu assertion nào, ngang sửa tự do, và không che lỗi nghiệp vụ nào (0/17), trong khi LLM sửa tự do che 8–10/17 lỗi bằng cách đổi giá trị kỳ vọng hoặc chèn bước. Chi phí là khoảng 11 đề xuất mỗi lần chạy phải chuyển lên người, trong đó 6 là escalate oan.
- RQ4: giao thức, hai bộ nhiệm vụ và bộ chấm tự động bằng mutation đã sẵn sàng; dữ liệu người tham gia là phần việc còn lại.
- Bản thân quá trình đánh giá đã tìm ra nhiều lỗi mà unit test không bắt được: prompt thiếu đặc tả trường URL của goto, Chromium treo khi tải trang qua HTTP/2, self-healing chỉ chữa một locator mỗi lượt, BVA thiếu ngữ cảnh hành động, và locator nth trả sai phần tử. Mỗi lỗi đều được ghi lại cùng cách phát hiện và cách sửa.

6.2 Hạn chế

- Dataset RQ1 nhỏ (15 tài liệu, 47 yêu cầu) và nhãn chưa được nhiều người gán độc lập.
- RQ2 trên 50 target demo, phần lớn luồng ngắn; snapshot 6.000 ký tự có thể không đủ cho trang lớn.
- RQ3 đo trên một ứng dụng nhỏ với 14 mutation do nhóm thiết kế; người duyệt là mô phỏng.
- RQ4 chưa có dữ liệu người tham gia ở thời điểm viết chuyên đề.
- DSL chưa có select, hover, nhấn phím, double click.
- Policy xếp mọi thay đổi bên trong assertion vào mức High, kể cả khi chỉ đổi locator mà giữ giá trị kỳ vọng; cách này an toàn nhưng tạo escalate oan khi giao diện đổi class hay id của phần tử được kiểm.

6.3 Hướng phát triển

- Tiến hành thí nghiệm RQ4 theo giao thức đã chuẩn bị, 4–6 người tham gia.
- Mở rộng dataset RQ1, cho nhiều người gán nhãn độc lập, tính kappa cho giám khảo LLM.
- Đánh giá lại các cải tiến (quy tắc sinh test, vòng lặp grounding, chữa theo lô) trên tập dữ liệu mới, không dùng để phát triển chúng.
- Chạy RQ2, RQ3 với nhiều model (Gemma local, GPT) để đo độ dao động giữa model.
- Tách mức rủi ro của assertion: đổi phần tử được kiểm mà giữ giá trị kỳ vọng có thể là Medium kèm bằng chứng grounding.
- Mở rộng DSL (select, hover, phím) và thêm Alembic khi schema DB bắt đầu thay đổi trên SQL Server thật.

TÀI LIỆU THAM KHẢO

- [1] Microsoft. Playwright Test Agents. <https://playwright.dev/docs/test-agents>, 2025.
- [2] Microsoft. Playwright: Codegen và Best Practices. <https://playwright.dev/docs/best-practices>, 2026.
- [3] Z. Yang, R. Huang, C. Cui, N. Niu, D. Towey. Requirements-Based Test Generation: A Comprehensive Survey. arXiv:2505.02015, 2025.
- [4] X. Teoh, Y. Lin, D.-M. Nguyen, R. Ren, W. Zhang, J. S. Dong. WebTestPilot. FSE 2026, arXiv:2602.11724, DOI 10.1145/3797115.
- [5] A. M. Sami, Z. Rasheed, M. Waseem, Z. Zhang, T. Herda, P. Abrahamsson. A Tool for Test Case Scenarios Generation Using Large Language Models. arXiv:2406.07021, 2024.
- [6] R. N. Joseph. Beyond LLM-Based Test Automation: Zero-Cost Self-Healing via DOM Accessibility Tree. arXiv:2603.20358, 2026.
- [7] S. Masuda, S. Kouzawa, K. Sezai, H. Suhara, Y. Hiruta, K. Kudou. Generating High-Level Test Cases from Requirements using LLM: An Industry Study. arXiv:2510.03641, 2025.
- [8] H. Lee. Practical Limits of Autonomous Test Repair: A Multi-Agent Case Study. arXiv:2605.01471, 2026.
- [9] T. Schlathölter. ReqToCode: Embedding Requirements Traceability as a Structural Property of the Codebase. arXiv:2603.13999, 2026.
- [10] B. Tang, K. G. Shin. Steward: Natural Language Web Automation. arXiv:2409.15441, 2024.
- [11] S. Bhatia, T. Gandhi, D. Kumar, P. Jalote. System Test Case Design from Requirements Specifications: Insights and Challenges of Using ChatGPT. arXiv:2412.03693, 2024.
- [12] A. Kluge, A. Stocco. Web Element Relocalization in Evolving Web Applications. arXiv:2505.16424, 2025.

PHỤ LỤC A: DANH SÁCH 50 TARGET RQ2

Bảng A.1 Catalog target RQ2 (datasets/reference/rq2_targets.json)

ID	Website	Loại	Các bước	Kết quả mong đợi
SD-01	SauceDemo	positive	Nhập username 'standard_user' và password 'secret_sauce'; Bấm nút Login	Đăng nhập thành công, URL là https://www.saucedemo.com/inventory.html
SD-02	SauceDemo	negative	Nhập username 'standard_user' và password 'wrong_password'; Bấm Login	Hiện thông báo lỗi chứa 'Username and password do not match'
SD-03	SauceDemo	negative	Nhập username 'locked_out_user' và password 'secret_sauce'; Bấm Login	Hiện thông báo chứa 'Sorry, this user has been locked out'
SD-04	SauceDemo	negative	Không nhập gì; Bấm nút Login	Hiện thông báo chứa 'Username is required'
SD-05	SauceDemo	negative	Nhập username 'standard_user', bỏ trống password; Bấm Login	Hiện thông báo chứa 'Password is required'
SD-06	SauceDemo	positive	Nhập username 'problem_user' và password 'secret_sauce'; Bấm Login	Đăng nhập thành công, URL là https://www.saucedemo.com/inventory.html
SD-07	SauceDemo	positive	Nhập username 'performance_glitch_user' và password 'secret_sauce'; Bấm Login	Đăng nhập thành công, URL là https://www.saucedemo.com/inventory.html
TI-01	The Internet (Heroku)	positive	Nhập username 'tomsmith' và password 'SuperSecretPassword!'; Bấm nút Login	Hiện thông báo chứa 'You logged into a secure area!'
TI-02	The Internet (Heroku)	negative	Nhập username 'baduser' và password 'badpass'; Bấm Login	Hiện thông báo chứa 'Your username is invalid!'
TI-03	The Internet (Heroku)	positive	Bấm nút 'Add Element'	Xuất hiện nút 'Delete' trên trang
TI-04	The Internet (Heroku)	positive	Bấm nút 'Start'	Sau khi tải xong hiện văn bản chứa 'Hello World!'
TI-05	The Internet (Heroku)	positive	Bấm nút 'Start'	Sau khi tải xong hiện văn bản chứa 'Hello World!'
TI-06	The Internet (Heroku)	positive	Mở trang	Trang hiện văn bản chứa 'This page returned a 200 status code'
TI-07	The Internet	positive	Mở trang	Trang hiện văn bản chứa

ID	Website	Loại	Các bước	Kết quả mong đợi
	(Heroku)			'This page returned a 404 status code'
PT-01	Practice Test Automation	positive	Nhập username 'student' và password 'Password123'; Bấm nút Submit	Hiện thông báo chứa 'Logged In Successfully'
PT-02	Practice Test Automation	negative	Nhập username 'incorrectUser' và password 'Password123'; Bấm Submit	Hiện thông báo chứa 'Your username is invalid!'
PT-03	Practice Test Automation	negative	Nhập username 'student' và password 'wrongPassword'; Bấm Submit	Hiện thông báo chứa 'Your password is invalid!'
OH-01	OrangeHRM Demo	positive	Nhập username 'Admin' và password 'admin123'; Bấm nút Login	Đăng nhập thành công, hiện tiêu đề 'Dashboard'
OH-02	OrangeHRM Demo	negative	Nhập username 'Admin' và password 'wrongpass'; Bấm Login	Hiện thông báo chứa 'Invalid credentials'
TI-08	The Internet (Heroku)	positive	Bấm nút 'Click for JS Alert' (hộp thoại alert được đóng)	Mục Result hiện văn bản chứa 'You successfully clicked an alert'
TI-09	The Internet (Heroku)	positive	Bấm nút 'Click for JS Confirm'; Hộp thoại xác nhận bị đóng (tương đương Cancel)	Mục Result hiện văn bản chứa 'You clicked: Cancel'
TI-10	The Internet (Heroku)	positive	Bấm nút 'Remove' ở mục Remove/add	Sau khi xử lý xong hiện thông báo chứa "It's gone!"
TI-11	The Internet (Heroku)	positive	Bấm nút 'Enable' ở mục Enable/disable	Sau khi xử lý xong hiện thông báo chứa "It's enabled!"
TI-12	The Internet (Heroku)	positive	Mở trang	Trang hiện văn bản chứa 'This page returned a 500 status code'
TI-13	The Internet (Heroku)	negative	Để trống username và password; Bấm nút Login	Hiện thông báo chứa 'Your username is invalid!'
TI-14	The Internet (Heroku)	negative	Nhập username 'tomsmith' và password 'wrongpass'; Bấm nút Login	Hiện thông báo chứa 'Your password is invalid!'
TI-15	The Internet (Heroku)	positive	Nhập username 'tomsmith' và password 'SuperSecretPassword!'; Bấm nút Login; Ở trang Secure Area bấm nút 'Logout'	Quay về trang login và hiện thông báo chứa 'You logged out of the secure area!'
TI-16	The Internet (Heroku)	positive	Bấm liên kết 'here'	Bị chuyển hướng, URL là https://the-internet.herokuapp.com/status/500

ID	Website	Loại	Các bước	Kết quả mong đợi
				tus_codes
SD-08	SauceDemo	positive	Đăng nhập với username 'standard_user' và password 'secret_sauce'; Bấm 'Add to cart' ở sản phẩm 'Sauce Labs Backpack'	Biểu tượng giỏ hàng hiện số '1'
SD-09	SauceDemo	positive	Đăng nhập với username 'standard_user' và password 'secret_sauce'; Bấm 'Add to cart' ở sản phẩm 'Sauce Labs Backpack'; Bấm 'Remove' ở cùng sản phẩm đó	Nút 'Add to cart' của 'Sauce Labs Backpack' hiện lại
SD-10	SauceDemo	positive	Đăng nhập với username 'standard_user' và password 'secret_sauce'; Bấm vào tên sản phẩm 'Sauce Labs Backpack'	Mở trang chi tiết sản phẩm, mô tả chứa 'carry.allTheThings()'
SD-11	SauceDemo	negative	Đăng nhập với username 'standard_user' và password 'secret_sauce'; Thêm 'Sauce Labs Backpack' vào giỏ; Mở giỏ hàng, bấm 'Checkout'; Bỏ trống First Name, nhập Last Name 'Nguyen' và Zip '70000'; Bấm 'Continue'	Hiện thông báo lỗi chứa 'Error: First Name is required'
SD-12	SauceDemo	positive	Đăng nhập với username 'standard_user' và password 'secret_sauce'; Thêm 'Sauce Labs Backpack' vào giỏ; Mở giỏ hàng, bấm 'Checkout'; Nhập First Name 'An', Last Name 'Nguyen', Zip '70000', bấm 'Continue'; Bấm 'Finish'	Hiện tiêu đề 'Thank you for your order!'
SD-13	SauceDemo	positive	Đăng nhập với username 'standard_user' và password 'secret_sauce'; Mở menu (nút 'Open Menu'); Bấm 'Logout'	Quay về trang đăng nhập, URL là https://www.saucedemo.com/
SD-14	SauceDemo	positive	Đăng nhập với username 'standard_user' và password 'secret_sauce'; Bấm biểu tượng giỏ hàng	Mở trang giỏ hàng, tiêu đề chứa 'Your Cart'
UP-01	UI Testing Playground	positive	Nhập User Name 'tester' và password 'pwd'; Bấm nút 'Log In'	Hiện trạng thái chứa 'Welcome, tester!'
UP-02	UI Testing Playground	negative	Nhập User Name 'tester' và password 'wrong'; Bấm nút	Hiện trạng thái chứa 'Invalid'

ID	Website	Loại	Các bước	Kết quả mong đợi
			'Log In'	username/password'
UP-03	UI Testing Playground	positive	Nhập User Name 'tester' và password 'pwd'; Bấm nút 'Log In'; Bấm nút 'Log Out'	Hiện trạng thái chứa 'User logged out.'
UP-04	UI Testing Playground	positive	Nhập 'Hello' vào ô 'Set New Button Name'; Bấm nút bên dưới ô nhập	Nút đổi tên thành 'Hello'
UP-05	UI Testing Playground	positive	Mở trang	Ở mục Playground hiện văn bản chứa 'Welcome UserName!'
UP-06	UI Testing Playground	positive	Bấm liên kết 'Click me' một lần	Bộ đếm hiện 'The link above clicked 1 times.'
UP-07	UI Testing Playground	positive	Ở trang chủ bấm liên kết 'Sample App'	Chuyển tới trang Sample App, URL là http://uitestingplayground.com/sampleapp
UP-08	UI Testing Playground	positive	Ở trang chủ bấm liên kết 'Dynamic ID'	Chuyển tới trang có tiêu đề 'Dynamic ID'
DQ-01	DemoQA	positive	Nhập Full Name 'Nguyen Van A' và Email 'a@example.com'; Bấm nút 'Submit'	Khung kết quả hiện 'Name:Nguyen Van A'
DQ-02	DemoQA	positive	Chọn radio 'Yes'	Hiện văn bản 'You have selected Yes'
DQ-03	DemoQA	positive	Chọn radio 'Impressive'	Hiện văn bản 'You have selected Impressive'
DQ-04	DemoQA	positive	Bấm (click đơn) nút 'Click Me'	Hiện văn bản 'You have done a dynamic click'
PT-04	Practice Test Automation	positive	Nhập username 'student' và password 'Password123'; Bấm nút Submit; Bấm nút 'Log out'	Quay về trang login, URL là https://practicetestautomation.com/practice-test-login/
PT-05	Practice Test Automation	positive	Bấm nút 'Add'	Hiện thông báo xác nhận chứa 'Row 2 was added'
OH-03	OrangeHRM Demo	negative	Nhập username 'Admin', bỏ trống password; Bấm nút Login	Hiện thông báo kiểm tra hợp lệ 'Required' dưới ô Password

PHỤ LỤC B: BỘ 20 TEST SHOPLAB (RQ3)

Bảng B.1 Bộ test ShopLab (apps/shoplab/suite.py)

ID	Loại	Tiêu đề	Các bước	Kết quả mong đợi
R01	positive	Đăng nhập hợp lệ	Đăng nhập bằng alice / secret123	Vào trang Products, URL là /products
R02	negative	Sai mật khẩu	Đăng nhập alice với mật khẩu 'wrong'	Hiện lỗi 'Invalid username or password'
R03	negative	Bỏ trống username	Bấm Login khi chưa nhập gì	Hiện lỗi 'Username is required'
R04	negative	Tài khoản bị khoá	Đăng nhập bob / secret123	Hiện lỗi 'Account is locked'
R05	positive	Đăng xuất	Đăng nhập bằng alice / secret123; Bấm Logout	Về trang đăng nhập, hiện 'You have been logged out'
R06	positive	Thêm 1 sản phẩm	Đăng nhập bằng alice / secret123; Thêm Sauce Backpack vào giỏ	Badge giỏ hàng hiện 1
R07	positive	Thêm 2 sản phẩm	Đăng nhập bằng alice / secret123; Thêm Sauce Backpack và Bike Light	Badge giỏ hàng hiện 2
R08	positive	Tổng tiền 2 món	Đăng nhập bằng alice / secret123; Thêm Sauce Backpack và Bike Light; Mở giỏ hàng	Tổng tiền là \$39.98
R09	positive	Giỏ hàng có món đã thêm	Đăng nhập bằng alice / secret123; Thêm Bolt T-Shirt; Mở giỏ hàng	Giỏ hàng có 'Bolt T-Shirt'
R10	positive	Giá sản phẩm đầu tiên	Đăng nhập bằng alice / secret123	Sản phẩm đầu có giá \$29.99
R11	positive	Checkout thành công	Đăng nhập bằng alice / secret123; Thêm Sauce Backpack; Mở giỏ, bấm Checkout; Nhập First name An, Last name Nguyen, Zip 70000; Bấm Continue	Hiện 'Thank you for your order!'
R12	negative	Checkout thiếu First name	Đăng nhập bằng alice / secret123; Thêm Sauce Backpack; Mở giỏ, bấm Checkout; Bỏ trống First name, nhập Last name và Zip; Bấm Continue	Hiện lỗi 'First name is required'
R13	positive	Tổng tiền 3 món	Đăng nhập bằng alice / secret123; Thêm cả 3 sản phẩm; Mở giỏ hàng	Tổng tiền là \$55.97

ID	Loại	Tiêu đề	Các bước	Kết quả mong đợi
R14	positive	Lưu hồ sơ	Đăng nhập bằng alice / secret123; Mở Profile; Đổi Display name thành 'Alice A'; Bấm Save	Hiện 'Profile saved'
R15	positive	Chuyển hướng sau đăng nhập	Đăng nhập bằng alice / secret123	URL là /products
R16	positive	Giỏ hàng trống	Đăng nhập bằng alice / secret123; Mở giỏ hàng	Hiện 'Cart is empty'
R17	positive	Danh sách sản phẩm có nút thêm giỏ	Đăng nhập bằng alice / secret123	Thấy nút thêm Sauce Backpack
R18	positive	Giỏ có nút Checkout	Đăng nhập bằng alice / secret123; Thêm Sauce Backpack; Mở giỏ hàng	Thấy nút Checkout
R19	negative	Người dùng không tồn tại	Đăng nhập mallory / secret123	Hiện lỗi 'Invalid username or password'
R20	positive	Đăng xuất về trang có nút Login	Đăng nhập bằng alice / secret123; Bấm Logout	Thấy nút Login

PHỤ LỤC C: ĐIỀU KIỆN KIỂM THỬ GOLD (RQ1)

Bảng C.1 83 điều kiện kiểm thử gold (*datasets/reference/ambiguity_eval/test_conditions.json*)

Tài liệu	ID	UC	Loại	Điều kiện
auth	A1	1	positive	Email và mật khẩu hợp lệ thì đăng nhập được và chuyển tới trang chủ
auth	A2	1	negative	Email hoặc mật khẩu sai thì hiển thị lỗi và không cho vào
auth	A3	2	positive	Đổi mật khẩu thành công thì hệ thống gửi email xác nhận
auth	A4	3	negative	Đăng nhập sai nhiều lần thì tài khoản bị khoá và người dùng được thông báo
booking	B1	1	positive	Tìm chuyến thì hiển thị các chuyến phù hợp
booking	B2	2	positive	Người dùng đã đăng nhập chọn chuyến và xác nhận thì được giữ chỗ và nhận mã đặt chỗ
booking	B3	2	negative	Người dùng chưa đăng nhập thì không đặt chỗ được
booking	B4	3	positive	Hủy đặt chỗ trước giờ khởi hành thì hủy được
booking	B5	3	negative	Hủy đặt chỗ sau giờ khởi hành thì không được
booking	B6	4	positive	Người dùng gửi được yêu cầu hoàn tiền
cart	C1	1	positive	Nhấn nút Thêm vào giỏ thì thao tác thêm sản phẩm được thực hiện
cart	C2	2	positive	Thanh toán giỏ hàng thì tạo đơn và gửi email xác nhận
cart	C3	3	positive	Đơn hàng đã đặt được hệ thống xử lý
profile	P1	1	positive	Người dùng đã đăng nhập xem được tên, email và số điện thoại của mình
profile	P2	1	negative	Người dùng chưa đăng nhập thì không xem được hồ sơ
profile	P3	2	positive	Cập nhật được thông tin cá nhân
search	S1	1	positive	Nhập từ khoá thì hiển thị kết quả khớp
search	S2	1	boundary	Có hơn 10 kết quả thì mỗi trang hiển thị tối đa 10
search	S3	2	positive	Thời gian phản hồi của tìm kiếm được kiểm tra
search	S4	3	positive	Xem được tất cả kết quả khớp trên một trang
library	L1	1	positive	Thành viên đang mượn dưới 5 cuốn thì mượn thêm được
library	L2	1	boundary	Đang mượn 4 cuốn thì mượn cuốn thứ 5 thành công
library	L3	1	boundary	Đã mượn 5 cuốn thì lượt mượn mới bị từ chối với 'Đã đạt giới hạn mượn'
library	L4	1	negative	Chưa đăng nhập thì không mượn được
library	L5	2	positive	Gia hạn lần đầu thì hạn trả cộng thêm 7 ngày
library	L6	2	negative	Gia hạn lần thứ hai cùng một cuốn thì bị từ chối
library	L7	3	positive	Trả sách sau hạn thì có tính phí
transfer	T1	1	positive	Chuyển số tiền trong khoảng 10.000–50.000.000 thì thành công
transfer	T2	1	boundary	Chuyển đúng 10.000 hoặc đúng 50.000.000 thì được chấp nhận
transfer	T3	1	boundary	Chuyển 9.999 hoặc 50.000.001 thì bị từ chối với 'Số

Tài liệu	ID	UC	Loại	Điều kiện
				tiền không hợp lệ'
transfer	T4	1	negative	Chưa đăng nhập thì không chuyển được
transfer	T5	2	positive	Nhập đúng OTP thì giao dịch được thực hiện
transfer	T6	2	boundary	Nhập sai OTP 3 lần thì giao dịch bị huỷ (sai dưới 3 lần thì chưa huỷ)
transfer	T7	3	boundary	Tổng chuyển trong ngày vượt 100.000.000 thì không được chuyển thêm
course	K1	1	positive	Đăng ký trong thời gian mở đăng ký thì lưu được
course	K2	1	negative	Ngoài thời gian mở đăng ký thì không đăng ký được
course	K3	1	boundary	Tổng 14 hoặc 24 tín chỉ thì lưu được
course	K4	1	boundary	Tổng 13 hoặc 25 tín chỉ thì không lưu được và báo lỗi
course	K5	2	positive	Huỷ được học phần đã đăng ký
course	K6	3	positive	Đăng ký vào lớp đã đủ sĩ số thì được đưa vào danh sách chờ
inventory	I1	1	positive	Nhập số lượng hợp lệ thì tồn kho tăng tương ứng
inventory	I2	1	boundary	Nhập đúng 1 hoặc 10.000 đơn vị thì được chấp nhận
inventory	I3	1	boundary	Nhập 0 hoặc 10.001 đơn vị thì bị từ chối
inventory	I4	2	positive	Tồn kho xuống thấp thì có cảnh báo
inventory	I5	3	positive	Xuất kho không vượt tồn thì thành công và tồn giảm tương ứng
inventory	I6	3	negative	Xuất quá số lượng tồn thì không được xuất
restaurant	R1	1	positive	Đặt bàn hợp lệ thì được xác nhận và nhận tin nhắn xác nhận
restaurant	R2	1	boundary	Đặt cho 1 người hoặc số người tối đa thì được chấp nhận
restaurant	R3	1	boundary	Đặt cho 0 người hoặc vượt số người tối đa thì bị từ chối
restaurant	R4	2	positive	Khách đổi được giờ đặt bàn
restaurant	R5	3	positive	Khách đến trễ quá ngưỡng thì đặt bàn bị huỷ
restaurant	R6	4	negative	Mâu thuẫn giới hạn số khách (20 ở UC-1, 10 ở UC-4) được phát hiện hoặc kiểm tra
wallet	W1	1	positive	Nạp trong khoảng hợp lệ thì số dư tăng và hiển thị 'Nạp tiền thành công'
wallet	W2	1	boundary	Nạp đúng 50.000 hoặc đúng 20.000.000 thì được chấp nhận
wallet	W3	1	boundary	Nạp 49.999 hoặc 20.000.001 thì bị từ chối
wallet	W4	1	negative	Chưa liên kết ngân hàng thì không nạp được
wallet	W5	2	positive	Rút được tiền từ ví về tài khoản ngân hàng
wallet	W6	3	positive	Lịch sử hiển thị giao dịch 90 ngày gần nhất, mới nhất ở trên
wallet	W7	3	boundary	Giao dịch cũ hơn 90 ngày không hiển thị
leave	V1	1	positive	Gửi đơn hợp lệ thì đơn được chuyển tới quản lý trực tiếp
leave	V2	1	boundary	Đơn 0,5 ngày hoặc 12 ngày thì gửi được
leave	V3	1	boundary	Đơn dưới 0,5 ngày hoặc trên 12 ngày thì bị từ chối

Tài liệu	ID	UC	Loại	Điều kiện
leave	V4	2	positive	Quản lý duyệt thì nhân viên nhận thông báo kết quả
leave	V5	2	negative	Quản lý từ chối thì nhân viên nhận thông báo kết quả
leave	V6	3	negative	Xin nghỉ quá số ngày phép còn lại thì không được
social	O1	1	positive	Đăng bài hợp lệ thì bài hiển thị ở đầu bảng tin
social	O2	1	boundary	Bài dài 1 hoặc 2.000 ký tự thì đăng được
social	O3	1	boundary	Bài rỗng hoặc dài 2.001 ký tự thì không đăng được
social	O4	1	negative	Chưa đăng nhập thì không đăng bài được
social	O5	2	positive	Đính kèm được ảnh vào bài viết
social	O6	3	positive	Bài viết bị báo cáo được đưa vào xem xét
coupon	Q1	1	positive	Mã hợp lệ thì tổng tiền giảm 10%
coupon	Q2	1	negative	Mã hết hạn thì hiển thị 'Mã đã hết hạn' và không giảm
coupon	Q3	2	boundary	Đơn đúng 200.000 đồng thì áp mã được
coupon	Q4	2	boundary	Đơn 199.999 đồng thì không áp mã được
coupon	Q5	3	positive	Khách nhập thông tin và nhận được ưu đãi thành viên
ticket	X1	1	positive	Tạo ticket hợp lệ thì có mã ticket và email xác nhận
ticket	X2	1	boundary	Tiêu đề 5 ký tự hoặc độ dài tối đa thì tạo được
ticket	X3	1	boundary	Tiêu đề 4 ký tự hoặc vượt độ dài tối đa thì không tạo được
ticket	X4	1	negative	Chưa đăng nhập thì không tạo ticket được
ticket	X5	2	positive	Nhân viên phản hồi được ticket
ticket	X6	3	positive	Đóng ticket đã giải quyết thì chuyển sang 'Đã đóng'
ticket	X7	4	negative	Mâu thuẫn độ dài tiêu đề (100 ở UC-1, 50 ở UC-4) được phát hiện hoặc kiểm tra

PHỤ LỤC D: HƯỚNG DẪN CHẠY LẠI

Cài đặt: `pip install -r requirements.txt` rồi `python -m playwright install chromium`. Cấu hình LLM trong `.env` (local) và `.env.cloud` (DeepSeek), xem `.env.example`; hai file này không được commit. Mọi lệnh chạy ở thư mục gốc dự án.

Bảng D.1 Lệnh tái lập các số liệu trong báo cáo

Mục	Lệnh
Unit test	<code>python -m pytest tests/unit -q</code>
Giao diện	<code>streamlit run src/ui/app.py</code>
ShopLab	<code>python apps/shoplab/app.py --variant v0 --port 5100</code>
RQ1 phát hiện mơ hồ	<code>python evaluate_agent.py --profile cloud</code>
RQ1 chất lượng test case	<code>python evaluate_rq1.py --profile cloud --out rq1_results_v2.json</code>
RQ1 kappa	<code>python evaluate_rq1.py --kappa rq1_judge_sample_v2.csv</code>
RQ2 kiểm catalog	<code>python run_rq2.py --context oracle</code>
RQ2 lần đo đầu	<code>python run_rq2.py --profile cloud --context both --out rq2_results_both.json</code>
RQ2 bản 2	<code>python run_rq2.py --profile cloud --context all --out rq2_v2_r1.json</code> (lặp r2, r3)
RQ3 kiểm suite	<code>python run_rq3.py --check</code>
RQ3 ba nhánh	<code>python run_rq3.py --profile cloud --out rq3_results_v2.json</code>
RQ4	<code>python tools/rq4/rq4.py timer export check analyze</code> (xem <code>rq4-protocol.md</code>)